

IP 설계 이론

혼성신호·디지털 IC 설계 — 대학원 이론서 · 한국어판
옮긴 장 5 / 23 · 아직 다 옮기지 않았다

차례

[T1. 네 가지 기계로서의 MOSFET](#)

[T2. 지연 — RC 하나에서 타이밍 경로 전체까지](#)

[T3. 준안정 — 설계로 없앨 수 없는 단 하나의 실패](#)

[T4. 잡음 — 그리고 그것이 왜 모든 것의 크기를 정하나](#)

[T5. 표본화 — 연속 신호가 수가 되는 자리](#)

T1. 네 가지 기계로서의 MOSFET

소자는 하나인데 동작 영역은 넷이다 — 그리고 영역마다 다른 회로를 설계하게 된다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 전기장 · 전하 · 전자 · 지수함수 · 로그 · 미분 · 옴의법칙

이 장에서 새로 나오는 것

문턱전압 · 차단영역 · 선형영역 · 포화영역 · 아문턱영역 · 온저항 · 아문턱기울기 · 누설전류 · 트랜스컨덕턴스 · 채널 길이변조 · 출력저항 · 바디효과 · 게이트용량

MOSFET 은 보통 스위치로 소개되고, 디지털 첫 수업에는 그것으로 충분하다. 그러나 양산 IP 를 설계하기에는 충분하지 않다. 같은 네 단자 소자가 걸어 준 전압에 따라 질적으로 다른 네 가지 기계로 동작하고, 일하는 설계자는 그 넷 사이를 오간다 — 흔히 한 블록 안에서 그러다. 표본화 전단은 이 소자를 저항으로 쓰고, 그 뒤의 비교기는 증폭기로 쓰며, 다시 그 뒤의 디지털 논리는 스위치로 쓴다. 그리고 그 전부가 놓고 있는 동안 전지를 비우는 누설은, 아무도 설계하지 않은 영역에서 소자가 아주 나쁜 스위치로 동작하는 것이다.

이 장은 네 영역을 세운 다음, 영역마다 어디에 쓰는지, 언제 꺼내 쓰는지, 어떻게 적용하는지, 그리고 실제 IP 납품 물에 팔려 나가는 코드가 어떻게 생겼는지를 적는다. 이 책의 뒤에서 쓰이지 않는 것은 여기에 넣지 않았다.

T1.1 게이트가 실제로 하는 일

영역으로 들어가기 전에 물리 사실 하나를 먼저 본다. 뒤의 모든 식이 이것 위에 서 있기 때문이다. 게이트는 축전기의 한쪽 판이다. 채널 — 산화막 아래의 얇은 실리콘 층 — 이 다른 쪽 판이다. 그 사이에 원자 몇 개 두께의 절연체가 있다.

단위 면적당 게이트 산화막 용량, $C_{ox} = \epsilon_{ox}/t_{ox}$. 물리적 산화막 두께가 1.2 nm 이고 $\epsilon_{ox} \approx 3.9 \epsilon_0$ 이면, 이 값은 약 16 mF/m^2 — 제곱 마이크로미터당 대략 $1.60 \times 10^{19} \text{ fF}$ 다. 트랜지스터가 하는 모든 일이 이 값에 비례한다.

유도 — 게이트 전압이 왜 전류를 제어하는가

	이렇게 된다	왜냐하면
1	NMOS의 게이트에 양의 전압을 건다. 게이트는 축전기의 한 쪽 판이므로 단위 면적당 전하 $Q = C_{ox} \cdot V$ 를 쌓는다.	그것이 용량의 정의다. 아직 트랜지스터 물리는 하나도 안 썼다.
2	반대 부호의 전하가 다른 쪽 판 — 산화막 바로 아래의 실리콘 — 에 나타나야 한다.	축전기는 두 판에 크기가 같고 부호가 반대인 전하를 갖는다.
3	p형 실리콘에서 움직일 수 있는 음전하는 전자인데, 전자는 귀하다. 게이트 전압이 어느 값 아래면 실리콘은 필요한 전하를 정공을 밀어내서 대신 마련하고, 그 자리에 고정된 음의 반개 이온이 남는다.	다수 운반자를 비우는 쪽이 소수 운반자를 만드는 쪽보다 싸다.
4	게이트 전압이 어느 값을 넘으면 공핍이 더 자라기를 멈추고, 실리콘이 대신 움직이는 전자를 내놓기 시작한다. 그 전자 층이 채널이다.	표면이 충분히 끌려가고 나면, 전자를 끌어오는 것이 공핍층을 더 넓히는 것보다 에너지가 덜 든다.
5	그 값임이 일어나는 게이트 전압이 문턱전압 V_t다.	소자에 설계해 넣은 스위치 동작점이 아니다. 실리콘이 어떤 종류의 전하를 내놓을지를 바꾸는 전압이다.
6	V_t 위에서 채널의 움직이는 전하는 단위 면적당 $Q_{ch} = C_{ox}(V_{GS} - V_t)$ 다.	처음 V_t 만큼의 게이트 전압은 공핍에 쓰였다. 그것을 넘은 몫만 움직이는 전하를 만든다.
7	전류는 전하 공급 속도이고, 속도는 이동도 곱하기 전기장이다: $I = W \cdot Q_{ch} \cdot \mu \cdot E$.	표류 전류다. 저항에서 옴의 법칙을 주는 바로 그 관계다.

마지막 줄이 이 소자의 전부다. 아래의 모든 식은 이 관계에, 전기장 E 가 채널을 따라 어떻게 변하는지에 대한 서로 다른 가정을 얹은 것이다. $V_{GS} - V_t$ 라는 양은 너무 자주 나와서 이름이 있다. **과구동(overdrive)** 이고 V_{ov} 로 적는다. 설계자가 “그 소자를 과구동 150 밀리볼트로 돌렸다”고 말할 때 그 수가 이것이다.

V_t 는 하나의 수가 아니다. 온도에 따라 움직이고 (대략 $-1 \text{ mV}/^\circ\text{C}$), 바디 바이어스에 따라, 채널 길이에 따라, 이웃한 폴리실리콘이 얼마나 가까운지에 따라 움직인다. 그리고 다이마다 다르고 같은 다이 안의 트랜지스터마다 다르다. 공칭 V_t 에서만 도는 설계는 도는 설계가 아니다. T12 장이 이것을 정량화한다. 지금은 이 장의 모든 V_t 를 분포의 중심이라고 여기라.

T1.2 영역 1 — 차단: 완전히 꺼지지 않는 스위치

차단과 아문턱 누설

문턱 아래에서 채널은 비어 있지 않다 — 지수적으로 희박할 뿐이다. 전류는 멈추지 않는다. V_{GS} 가 V_t 아래로 **85 mV** 내려갈 때마다 열 배씩 준다. 그 기울기가 **아문턱 기울기 S** 이고, 저전력 설계에서 가장 중요한 수 하나다.

$$I_{leak} = I_0 \cdot 10^{(V_{GS} - V_t)/S}$$

S 에는 바닥이 있다. 채널 전하는 볼츠만 인자를 거쳐 표면 전위에 반응하므로 상온에서 $S \geq (kT/q) \cdot \ln 10 = \mathbf{59.5 \text{ mV/decade}}$ 이고, 실제 소자는 게이트 전압의 일부가 공핍 용량에 새기 때문에 그보다 20-40 % 위에 앉는다. **이 바닥이, 전원 전압이 줄어드는 동안 누설이 전력의 지배항이 된 까닭이다**: 속도를 지키려고 V_t 를 낮추면 누설을 지수로 물어야 하고, 그 지수를 더 가파르게 만들 수는 없다.

어디에 쓰나	내보내는 모든 블록의 대기 전력. 보존 레지스터. 메모리 배열(누설이 셀 수백만 개로 곱해지는 자리). 전원 게이팅과 헤더/푸터 스위치의 치수 결정. 대기 전류를 명시해서 파는 모든 IP — 대개가 그렇다.
언제 쓰나	데이터시트에 누설 수치를 적을 때마다. 합성에서 소자 종류를 고를 때마다. 고객이 보존 모드를 요구할 때마다. 그리고 블록이 대부분 놓고 있을 때마다 — 시간의 3 % 만 도는 등화기는 동적 전력 문제가 아니라 누설 문제다.
어떻게 쓰나	파운드리 특성화에서 최악 코너(최고 온도, 최저 V_t , 최고 전원 — 누설은 속도와 반대 방향으로 나빠진다)의 폭 마이크로미터당 I_{off} 를 받는다. 블록의 총 폭을 곱한다. 온도 인자를 다시 곱한다 — 경험칙은 10 °C 마다 2 배이고, S 자체가 T 에 비례하므로 S 식에서 바로 따라 나온다. 그리고 나서 정한다: 답이 예산을 넘으면 임계가 아닌 경로를 고 V_t 셀로 옮기고 타이밍을 다시 잡는다.

실제 코드는 이렇게 생겼다

```
// Liberty (.lib) — what the foundry actually ships you.
// This is the leakage the synthesiser reads. Note that it is
// state-dependent: a NAND2 leaks differently per input pattern,
// because the stacked device changes the internal node voltage.

cell (NAND2_X1) {
    area : 1.596 ;
    cell_leakage_power : 4.2851e+03 ;          // nW, the default
    leakage_power () {
        when      : "!A1 & !A2" ;
        value      : 1.8032e+03 ;          // both off: stack effect helps
    }
    leakage_power () {
        when      : "A1 & !A2" ;
        value      : 4.9210e+03 ;
    }
    leakage_power () {
        when      : "A1 & A2" ;
        value      : 6.1104e+03 ;          // both on: worst
    }
    ...
}

# Synthesis — trade leakage against timing by device flavour.
# The tool will use LVT only where it must to close timing.
set_db library [list sc9_base_svt.lib sc9_base_hvt.lib sc9_base_lvt.lib ]
set_db leakage_power_effort      high
set_db dynamic_power_effort      medium
# Forbid the leaky flavour outside the critical paths:
set_dont_use [get_lib_cells */*_LVT*]
# ... place and route, then selectively re-enable on failing paths:
set_dont_use -false [get_lib_cells */BUF_X*_LVT]
```

여기서 틀린다. 공칭 온도에서 누설을 인용하는 것. 누설은 10 °C 마다 대략 두 배가 되므로, 25 °C 수치는 125 °C 차량용 코너에 대고 보면 **8-16 배 낙관적**이다. 온도와 전원과 코너를 말하지 않고 “대기 12 μA ” 라고 적은 데이터시트는 규격이 아니다 — 그리고 자기 열 한계에서 진짜 수를 알게 된 고객은 다시 사지 않는다.

예제 6. 데이터시트가 요구하는 방식으로 낸 블록의 누설

주어진 것. 게이트 40,000 개, 게이트당 평균 소자 폭 0.8 μm (n 과 p 를 합쳐), 25 °C 전형 코너에서 $V_{GS} = 0$ 일 때 $I_{off} = 0.01 \text{ nA}/\mu\text{m}$ 인 블록. 고객은 105 °C 에서 돌리며 최악 대기 전류를 묻는다.

푸는 법. 총 폭 곱하기 마이크로미터당 I_{off} , 곱하기 온도 인자(10 °C 마다 2 배), 곱하기 fast-fast 실리콘의 코너 인자 (누설에서 보통 2-3 배).

답. 폭 = $40,000 \times 0.8 = 32,000 \mu\text{m}$. 25 °C 전형에서 $32,000 \times 0.01 \text{ nA} = \mathbf{0.24 \mu\text{A}}$. 온도: $105 - 25 = 80$ °C 이므로 $2^8 = 256$ 배. 코너: $\times 2.5$. 합하면 약 **0.16 mA**.

10 °C 마다 2 배라는 규칙은 지수를 어림한 것이다. 진짜 납품물에서는 외삽하지 않고 파운드리 코너 표에서 수를 읽는다. 이 어림의 쓸모는, 예산의 두 배 안에 있는지 십 배 밖에 있는지를 **이른 시점에** 아는 것이다.

여기서 틀린다. 여기서 25 °C 전형 수치와 105 °C fast 수치는 **640 배** 차이가 난다. 기술자가 앞의 것을 인용하는 까닭은 도구가 기본으로 찍어 주는 수가 그것이기 때문이다. 고객이 재는 수는 뒤의 것이다. 수 옆에 늘 세 가지 — 온도, 전원, 공정 코너 — 를 같이 적으라. 그것이 없는 누설 수치는 아무 뜻이 없다.

T1.3 영역 2 — 선형: 저항으로서의 트랜지스터

선형(삼극관) 영역과 온저항

게이트를 V_t 보다 한참 위로 올리고 드레인을 소스 가까이 붙들어 둔다. 이제 채널이 길이 전체에 걸쳐 존재하고 그 위의 전기장이 거의 고르므로, 소자는 게이트가 값을 제어하는 저항처럼 동작한다:

$$I_D = \mu C_{ox}(W/L)[(V_{GS} - V_t)V_{DS} - V_{DS}^2/2]$$

V_{DS} 가 작으면 제곱 항이 사라지고 소자는 그냥 이것이다

$$R_{on} = 1 / [\mu C_{ox}(W/L)(V_{GS} - V_t)]$$

저 분모를 읽으라. R_{on} 은 과구동에 반비례한다. 지나보내는 신호가 V_S 를 움직이면 V_{GS} 도 따라 움직이고, **저항이 신호에 따라 변한다.** 표본화 경로에서 신호에 따라 변하는 저항은 왜곡이고, 부트스트랩 스위치가 존재하는 까닭이 그것이다.

어디에 쓰나

모든 전송 게이트와 패스 게이트. 모든 ADC 와 모든 트래킹앰플리퍼 앞단의 표본화 스위치. 전원 게이팅 헤더와 푸터. CTLE 안의 가변 저항. SRAM 의 쓰기 드라이버. 디지털 신호를 다시 만들지 않고 지나보내야 하는 모든 자리.

언제 쓰나

한 마디를 구동하는 것이 아니라 두 마디를 잇는 것이 필요할 때. “신호를 버퍼로 받는다” 가 아니라 “신호를 지나보낸다” 라고 쓰고 있는 자신을 발견하는 순간, 선형 영역에 있고 R_{on} 이 문제가 된다.

어떻게 쓰나

정착을 먼저 예산 잡고 거기서 치수를 낸다. 저항 R 인 스위치가 용량 C 를 채우면 시간상수 $\tau = RC$ 로 정착한다. N 비트까지 정착하려면 $t_{settle} \geq RC \cdot \ln(2^{N+1})$ 이 필요하다. R 로 풀고, $R_{on} = 1/[\mu C_{ox}(W/L)V_{ov}]$ 로 W/L 을 낸다. 그다음 **최악 신호 준위에서 R_{on} 을 확인한다** — 중간 눈금이 아니다. 올라가는 신호를 지나보내는 NMOS 는 신호가 오를수록 V_{GS} 가 줄고, $V_{DD} - V_t$ 근처에서 R_{on} 이 터진다.

실제 코드는 이렇게 생겼다

```

* SPICE — the measurement you actually run on a sampling switch.
* You are not checking that it works; you are checking how Ron varies
* with the signal, because that variation is the distortion.

.param vin_val=0.45
M1 drain gate source bulk nch W=2u L=30n
Vg gate 0 DC 0.9
Vs source 0 DC 'vin_val'
Vd drain 0 DC 'vin_val+1m' $ 1 mV across the switch: stay in triode
.dc vin_val 0 0.9 0.01
.measure dc ron PARAM='1m/I(Vd)' $ R = V/I at each input level
.alter
.param boot=1 $ repeat with a bootstrapped gate
.end

// Verilog-AMS behavioural model used in mixed-signal top-level sims.
// A digital verification engineer meets this, not the SPICE deck.
module sw_nmos (inout d, inout s, input g);
    parameter real ron_nom = 250.0; // ohms at mid-scale
    parameter real roff = 1e12;
    analog begin
        // Ron rises as (Vgs - Vt) falls: this is the term that matters.
        real vgs, ron;
        vgs = V(g) - V(s);
        ron = (vgs > 0.35) ? ron_nom * 0.55 / (vgs - 0.35) : roff;
        I(d,s) <+ V(d,s) / ron;
    end
endmodule

```

여기서 틀린다. 중간 눈금에서 스위치 치수를 정하고 끝났다고 하는 것. 입력 범위 꼭대기에서의 R_{on} 은 중간 눈금 값의 몇 배가 될 수 있고, 그것이 만드는 정착 오차는 신호에 따라 달라진다 — 즉 잡음이 아니라 고조파 왜곡이고, 평균을 내도 없어지지 않는다. 입력 범위 전체에 대해 R_{on} 을 살펴 보고 가장 나쁜 점에 맞춰 치수를 정하라.

예제 7. 정착 요구에서 표본화 스위치의 치수를 낸다

주어진 것. 10 비트 변환기가 $C = 1$ pF 에 2 ns 안에 표본화한다. 전원 0.9 V, $V_t = 0.3$ V, 최악 입력이 0.5 V 이므로 $V_{ov} = 0.9 - 0.5 - 0.35 = 0.1$ V.

푸는 법. 10 비트까지 정착하려면 오차가 $\frac{1}{2}$ LSB 아래여야 하므로 $e^{-t/\tau} < 2^{-11}$, 즉 $t/\tau > 11 \cdot \ln 2 = 7.62$ 다. 경로의 나머지를 위해 τ 에 2 배 여유를 두고, $\tau = RC$ 에서 R 을 풀고, R_{on} 에서 W/L 을 낸다.

답. $\tau \leq 2$ ns / 7.62 = 262 ps. 2 배 여유를 두면 $\tau = 131$ ps. $R = \tau/C = 131$ Ω . 그러면 $W/L = 1/(\mu C_{ox} \cdot R \cdot V_{ov}) = 317$.

여기서 틀린다. 최악 입력이 아니라 중간 눈금 입력(0.45 V)에서 V_{ov} 를 잡으면 $V_{ov} = 0.1$ V 가 나오고 W/L 이 약 2 배 작아진다 — 그러면 스위치는 신호가 가장 큰 바로 그 자리에서 정착 목표를 놓치고, 왜곡이 가장 잘 보이는 자리가 거기다. 스위치는 평균 신호가 아니라 최악 신호 준위에서 치수를 정하라.

V_{DS} 를 과구동보다 높이면 드레인 근처에서 채널이 핀치오프 된다. 그 자리의 국소 V_{GS} 가 V_t 까지 떨어져 더 이상 움직이는 전하가 없다. 그 점을 넘으면 드레인 전압을 올려도 전류가 별로 안 는다. 소자가 게이트로 제어 되는 전류원이 된 것이다:

$$I_D = \frac{1}{2} \mu C_{ox} (W/L) (V_{GS} - V_t)^2 \cdot (1 + \lambda V_{DS})$$

여기서 나오는 두 양이 아날로그 설계의 일을 다 한다:

트랜스컨덕턴스 $g_m = \partial I_D / \partial V_{GS} = 2I_D / V_{ov}$. 입력 1 볼트당 얼마나 많은 전류가 나오나 — 즉 이득을 만드는 항이다.

출력저항 $r_o = \partial V_{DS} / \partial I_D \approx 1/(\lambda I_D)$. 이상적인 전류원에 얼마나 가까운가. λ 라는 항이 있는 까닭은 핀치오프 지점이 드레인 전압에 따라 움직이기 때문이고 — **채널 길이 변조** — 짧은 채널일수록 나쁘다. 아날로그 블록 이 최소 길이의 몇 배 되는 소자를 쓰는 까닭이 그것이다.

그 곱 $g_m r_o = 2/(\lambda V_{ov})$ 가 고유 이득이다: **트랜지스터 하나가 줄 수 있는 전압 이득의 최대치**. 공정이 작아질 수록 이 값이 줄고, 그래서 요즘 아날로그 설계는 날 이득보다 디지털의 도움에 기댄다.

어디에 쓰나	비교기와 감지증폭기 — 그 앞단 증폭기. 모든 전류 거울과 바이어스 회로. CTLE 와 SerDes 의 아날로그 전단. 천이 중의 레벨 시프터. PLL 의 링 발진기 지연 셀. 회로가 스위칭 이 아니라 증폭 을 해야 하는 모든 자리.
언제 쓰나	물음이 “이득이 얼마인가” 또는 “부하와 무관하게 전류가 얼마인가” 일 때. 그리고 뜻밖에도 디지털 분석에서도 쓴다: 논리 천이 동안 두 소자가 모두 포화를 지나가고, 그때 흐르는 단락 전류가 T4 가 느린 입력 예지에 왜 전력이 드는지 설명할 때 쓰는 항이다.
어떻게 쓰나	W/L 이 아니라 g_m/I_D 로 설계한다. 필요한 g_m 을 이득이나 잡음 요구에서 고르고, g_m/I_D 를 속도와 전력의 맞바꿈에서 고른다 (g_m/I_D 가 크면 $\approx$ 과구동이 작고 $\approx$ 효율적이지만 느리다. g_m/I_D 가 작으면 $\approx$ 과구동이 크고 $\approx$ 빠르지만 전력을 먹는다). 그러면 I_D 가 정해지고, W/L 은 파운드리가 특성화해 준 g_m/I_D 곡선에서 읽는다. 이 방법은 공정이 바뀌어도 통하고, 100 nm 아래에서 정확하기를 그만두는 제품 법칙 식에 기대지 않는다.

실제 코드는 이렇게 생겼다

- * The gm/ID sweep every analog team runs once per process,
- * then reuses for every block. You are not simulating a circuit here;
- * you are characterising the device so design becomes look-up.

```
.param L_val=100n
M1 d g 0 0 nch W=10u L='L_val'
Vg g 0 DC 0.5
Vd d 0 DC 0.45          $ hold Vds at VDD/2, in saturation
.dc Vg 0.2 0.9 0.005
.measure dc gm_id PARAM='deriv(I(Vd),V(g))/I(Vd)'
.measure dc ft PARAM='deriv(I(Vd),V(g))/(2*3.14159*cgg)'
.step param L_val list 30n 60n 100n 200n 500n
.end

// What a digital designer sees of all this: a characterised cell.
// The analog block arrives as a .lib timing/power view plus a
// behavioural model. You must still know what is inside, because
// the constraints you write depend on it.
cell (CMP_STRONGARM) {
    pin (CLK) { direction : input; clock : true; capacitance : 4.8; }
    pin (VINP) { direction : input; capacitance : 12.1; }
    pin (OUTP) { direction : output;
        timing () {
            related_pin : "CLK" ;
            timing_type : rising_edge ;
            cell_rise (delay_template) { ... }
        } }
    // The number a digital designer must not ignore:
    // the comparator has a metastability window like any bistable.
    // T3 derives it; the vendor states it here.
}
```

여기서 틀린다. 요즘 공정에서 제곱 법칙 식으로 설계하는 것. 대략 100 nm 아래에서는 운반자 속도가 포화하고 지수가 2 에서 1 쪽으로 밀려서, 식이 수십 퍼센트 틀릴 수 있다. 따져 보는 데는 여전히 훌륭하다 — 방향과 민감도를 바르게 알려 준다 — 그러나 수로는 쓸모가 없다. 식으로 따지고, 치수는 특성화 곡선에서 내려.

아문턱 동작

누설을 만드는 그 지수가 쓸 만한 전달 특성이기도 하다. V_t 아래로 바이어스하면 소자는 g_m/I_D 를 최대로 준다 — 약 27.1 V^{-1} , 오로지 S 가 정하는 값 — 이것이 어떤 MOSFET 도 넘지 못하는 전류 효율의 한계다. 값은 속도다. f_T 가 자릿수로 떨어지고, V_t 에 대한 지수 의존 때문에 소자 사이의 V_t 불일치가 수십 퍼센트의 전류 불일치로 바뀐다.

그러므로 아문턱은 연산당 에너지가 지배하고 속도는 그렇지 않을 때 고르는 영역이고, 전원이 꺼질 때 떠밀려 들어가는 영역이다. 또한 **보존** 회로가 사는 영역이다: 겨우 켜져 있는 소자가 상태를 붙들고 있다.

어디에 쓰나	상시 전원 영역 — 깨우기 논리, 워치독 타이머, 보존 레지스터. 센서 전단. 아주 작은 전류에서도 살아남아야 하는 바이어스 발생기. 그리고 반대 방향으로, 내보내는 모든 블록의 모든 누설 경로.
언제 쓰나	동작률이 아주 낮고 에너지 예산이 초당이 아니라 연산당일 때. 고객이 마이크로암페어 수치를 명시한 보존 모드를 요구할 때. V_t 쪽으로 무너지는 전원에서 살아남으라고 할 때.
어떻게 쓰나	속도가 아니라 정합이 면적을 정한다는 것을 받아들인다. 강반전에서는 전류 불일치가 g_m/I_D 를 거쳐 V_t 불일치를 따라간다. 아문턱에서는 g_m/I_D 가 최대이므로 같은 V_t 불일치가 가장 큰 전류 불일치를 만든다. 정합 요구에서 소자 치수를 내고(T12 의 펄그롬 관계), 그다음에 속도를 확인한다 — 반대 순서가 아니다.

실제 코드는 이렇게 생겼다

```
# UPF (Unified Power Format) — how the always-on domain is
# declared to the tools. A digital IP deliverable that supports
# retention ships this file, and it is as much a part of the
# product as the RTL.

create_power_domain PD_ALWAYS_ON -elements {u_wake u_timer}
create_power_domain PD_CORE -elements {u_dsp u_eq} -scope .

create_supply_net VDD_AON -domain PD_ALWAYS_ON
create_supply_net VDD_CORE -domain PD_CORE
set_domain_supply_net PD_CORE -primary_power_net VDD_CORE \
    -primary_ground_net VSS

# Retention: state survives while VDD_CORE collapses.
set_retention RET_CORE -domain PD_CORE \
    -retention_power_net VDD_AON -retention_ground_net VSS \
    -save_signal {u_dsp/ret_save high} \
    -restore_signal {u_dsp/ret_restore low}

# Isolation: outputs of a collapsed domain must not float into
# the always-on domain, or the always-on logic burns crowbar current.
set_isolation ISO_CORE -domain PD_CORE \
    -isolation_power_net VDD_AON -clamp_value 0 \
    -applies_to outputs
set_isolation_control ISO_CORE -domain PD_CORE \
    -isolation_signal u_pmc/iso_en -isolation_sense high
```

여기서 틀린다. 아이솔레이션을 잊는 것. 전원 영역이 무너지면 그 출력은 중간 전압으로 떠간다. 그것을 상시 전원 게이트에 넣으면 두 소자가 모두 도통한다 — 상시 전원 영역이 마이크로암페어가 아니라 밀리암페어를 먹게 되고, 팔았

던 보존 모드는 존재하지 않게 된다. 저전력 IP 에서 가장 흔한 통합 실패 가운데 하나이고, RTL 시뮬레이션에는 중간 전압이 없으므로 RTL 에서는 안 보인다.

T1.6 무시할 수 없는 이차 효과 둘

바디 효과

V_t 는 소스를 벌크에 묶은 상태로 적힌 값이다. 소스를 벌크보다 올리면 채널 아래 공핍층이 넓어지고, 반전이 일어나기까지 게이트 전압이 더 필요해진다. 즉 V_t 가 오른다. 의존 관계는 $V_t = V_{t0} + \gamma(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F})$ 이고, γ 는 0.2-0.5 $\sqrt{V}$ 정도의 공정 상수다.

실제로 따라오는 결과는 이것이다: **소스가 레일에 있지 않은 소자는 라이브러리가 말하는 것보다 약하다.**

어디에 쓰나	NAND/NOR 안의 쌓인 소자 — 위쪽 NMOS 는 소스가 접지보다 위에 있다. 패스 게이트 사슬. 캐스코드 전류원. 전송 게이트로 지은 다중화기 나무. 트랜지스터가 직렬로 놓이는 모든 자리.
언제 쓰나	쌓을 때마다. 4 입력 NAND 는 인버터보다 그냥 4 배 느린 것이 아니다 — 그보다 나쁘다. 위쪽 소자들이 바디 효과를 받기 때문이다. 넓은 게이트를 평평하게 짓지 않고 나무로 쪼개는 까닭 가운데 하나가 이것이다.
어떻게 쓰나	손으로 따질 때는 V_t 이동을 더하고 위쪽 소자의 과구동을 다시 계산한다. 실제 치수는 특성화된 라이브러리가 처리하게 둔다 — .lib 의 지연표에 이미 이 효과가 들어 있다 — 그러나 그것이 거기 있다는 것은 알고 있어야 , 새로 지어낸 쌓기 구조가 선형으로 비례할 것이라고 넘겨짚지 않는다. 쌓기가 깊으면 대신 버퍼를 끼운다.

실제 코드는 이렇게 생겼다

```
// Why you will not find `body effect` in an RTL file, but will
// feel it: the synthesiser's cell choice reflects it.
// These two are logically identical and time very differently.

assign y = ~(a & b & c & d);           // maps to NAND4 — deep stack

wire ab = ~(a & b);                   // maps to NAND2 + NAND2 + NOR2
wire cd = ~(c & d);                   // shallower stacks, more cells
assign y = ~(~ab | ~cd) ? 1'b0 : 1'b1;

// Liberty shows the penalty directly. Compare the intrinsic delays:
//  NAND2_X1  cell_rise (A1) ~ 0.021 ns
//  NAND3_X1  cell_rise (A1) ~ 0.031 ns
//  NAND4_X1  cell_rise (A1) ~ 0.045 ns    <- worse than linear
```

여기서 틀린다. 소자 n 개를 쌓으면 n 배 느려진다고 여기는 것. 바디 효과와 내부 마디 용량 때문에 초선형이고, 서넛을 넘으면 벌점이 빠르게 커진다. 합성된 넷리스트의 임계 경로에 NAND4 가 있다면, 대개 가장 먼저 구조를 뜯어고칠 자리가 거기다.

게이트 용량 — 당신이 몰고 있는 부하

당신이 모는 게이트는 축전기다: $C_g \approx C_{ox} \cdot W \cdot L$, 여기에 L 에 비례하지 않는 겹침 용량이 더해진다. 1.2 nm 산화막이면 게이트 면적당 $1.60 \times 10^7 \text{ fF}/\mu\text{m}^2$ 정도다.

이 사실 하나가 디지털 설계 전체를 묶는다. 한 게이트의 출력이 다음 게이트의 입력 용량을 채워야 하므로, **지연은 부하 용량 대 구동 세기의 비에 비례한다**. 그 비가 T_2 의 전기적 노력이고, 치수 정하기가 게이트마다의 문제가 아니라 사술의 문제인 까닭이 그것이다.

아날로그와 고속 디지털에서 무슨 향이 하나 더 있다. 게이트-드레인 겹침 용량 C_{gd} 가 출력 쪽에 그 단의 이득만큼 곱해져서 나타난다(밀러 효과). 비교기 앞단 증폭기에서는 이것이 부하를 지배할 수 있고, 디지털 게이트에서는 천이 동안 보이는 입력 용량이 정적인 값보다 큰 까닭이 된다.

어디에 쓰나	모든 지연 계산. 클럭 트리 전력(클럭은 엄청난 총 게이트 용량을 모는데, 클럭 게이팅이 첫 번째 전력 최적화인 까닭이 그것이다). 고객이 자기 드라이버를 예산 잡을 수 있도록 데이터시트에 적는 입력 용량.
언제 쓰나	디지털에서는 늘. 특히 IP 데이터시트의 입력 용량 줄을 적을 때, 그리고 셀 하나가 부하를 몇 개까지 몰아도 되는지 정할 때.
어떻게 쓰나	부하를 최소 인버터 입력 용량의 배수로 적으면 셈이 손으로 할 만해진다 — 논리적 노력(T_2)이 형식화한 것이 정확히 이것이다. 납품물 쪽에서는 핀마다의 입력 용량을 .lib 와 데이터시트에 적는다. 통합하는 쪽이 그것으로 자기 드라이버 치수를 정하고 클럭 트리를 짓는다.

실제 코드는 이렇게 생겼다

```
// The datasheet line that comes from this, and the .lib it matches.
// If these two disagree, the customer's timing closure fails and
// they will find out before you do.

pin (D) {
    direction      : input ;
    capacitance     : 1.412 ;      // fF — what the integrator budgets
    rise_capacitance : 1.455 ;
    fall_capacitance : 1.369 ;
    max_transition  : 0.300 ;      // ns — beyond this, delay tables
                                // are extrapolated, not valid
}

// SDC the customer writes against your block:
set_load      0.0014 [get_ports eq_out*] ;# pF
set_driving_cell -lib_cell BUF_X4 [get_ports adc_data*]
set_max_transition 0.300 [current_design]
```

여기서 틀린다. 넷이 max_transition 을 넘게 두는 것. 그 점을 넘으면 라이브러리의 지연표는 외삽되고 있는 것이고, 타이밍 보고서는 더 이상 특성화된 자료가 아니다 — 자신만만한 수가 옆에 붙은 짐작이다. max-transition 위반은 경고가 아니라 확실한 오류로 다루라.

T1.7 어느 일에 어느 영역인가

표 46. 네 영역은 네 가지 설계 연장이다

영역	조건	소자가 이렇게 동작한다	이런 데 쓴다	중요한 수
차단	$V_{GS} < V_t$	새는 열린 스위치	대기, 보존, 전원 게이팅	I_{off} 와 아문턱 기울기 S
선형	$V_{GS} > V_t, V_{DS} < V_{ov}$	게이트로 제어되는 저항	패스 게이트, 표본화 스위치, 전원 스위치	R_{on} , 그리고 그것이 신호에 따라 얼마나 변하나
포화	$V_{GS} > V_t, V_{DS} > V_{ov}$	전압으로 제어되는 전류원	증폭기, 비교기, 전류 거울, CTLE	$g_m, r_o, g_m/I_D$
아문턱	V_{GS} 가 V_t 보다 조금 < 작다	지수적 트랜스컨덕터	상시 전원 블록, 초저전력, 보존	S , 그리고 V_t 불일치

디지털 셀은 거의 모든 시간을 앞의 두 줄에서 보내고, 천이할 때마다 세 번째 줄을 잠깐 지나간다. 그 잠깐이 사소한 것이 아니다. 그것이 T2 의 단락 전류이고 T3 의 재생이며, 이 책에서 가장 어려운 문제 둘 — 준안정과 비교기의 판별 — 은 전부 소자가 선형이나 차단으로 정착하지 않고 포화에 머무는 동안 무슨 일이 일어나는가에 대한 논의다.

실제로 난 일

양산에 들어간 표본화 스위치를 중간 눈금 R_{on} 에서 치수 잡았고, 모든 코너의 시뮬레이션에서 정착 목표를 맞췄다. 그런데 실리콘에서 2 차 고조파 왜곡이 시뮬레이션보다 12 dB 나쁘게 나왔다. 까닭은 검증 쓸기가 정착을 볼 때는 입력을 중간 눈금에 붙들어 두고, 입력을 쓸 때는 이득만 봤다는 것이다 — 그래서 정착 중요한 조합, 큰 입력과 빠듯한 정착 이 한 번도 시뮬레이션되지 않았다. 설계에는 틀린 것이 없었다. **검증 격자**가 틀렸다. T21 이 이것을 규칙으로 만든다: 축을 하나씩 쓸지 말고 같이 쓸라.

표 47. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
아문턱 기울기 S	모든 블록의 대기 규격	데이터시트의 누설 수치, 그리고 보존이 가능한가
R_{on} 과 그 신호 의존	표본화 스위치, 패스 게이트, 전원 스위치	정착 시간, 왜곡, 전원 스위치 양단의 IR 강하
g_m/I_D	비교기 앞단, CTLE, 바이어스	전류 한 단위당 이득과 잡음 — 즉 전력 예산
채널 길이 변조 λ	전류 거울, 캐스코드	바이어스 전류가 출력 전압에 따라 얼마나 움직이나
바디 효과 γ	쌓인 논리, 전송 게이트 나무	깊은 쌓기가 왜 초선형으로 느린가
게이트 용량 C_g	모든 지연 계산과 클럭 전력 계산	데이터시트의 입력 용량 줄

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

소자 수준의 발명은 팹리스 IP 회사에는 대체로 닫혀 있다 — 소자는 파운드리인 것이다. **열려 있는 것은 주어진 소자들 가운데 무엇을 어떻게 고르는가다.** 실제로 팔리는 IP 포트폴리오의 청구항은 흔히 선택과 배치 에 걸린다: 상태를 붙드는 마디에만 고 V_t 소자를 쓰고 읽기 경로는 표준 V_t 두는 보존 래치. 코너 전체에서 경합 전류가 묶이도록 풀업 쌓기의 치수를 정한 레벨 시프터. V_{GS} 를 일정하게 붙들어 R_{on} 이 신호와 무관해지게 만드는 부스트스트랩 회로를 가진 표본화 스위치. 셋 다 소자 영역에 대한 논의이고, 셋 다 회로로서 특허가 된다. 나는 지금 어느 영역에 있고, 그 영역이 내게 무엇을 주는가를 물으며 이 장을 읽으라 — 청구항은 그 물음에서 나온다.

T2. 지연 — RC 하나에서 타이밍 경로 전체까지

도구가 찍어 주는 지연 수치는 넷 중 하나의 모형이다. 어느 것인지 알라. 모르면 당신 회로와 무관했던 수를 믿게 된다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 미분 · 지수함수 · 옴의법칙 · 문턱전압 · 선형영역 · 포화영역 · 온저항 · 게이트용량 · 출력저항

이 장에서 새로 나오는 것

RC지연 · 엘모어지연 · 논리적노력 · 전기적노력 · 기생지연 · FO4 · 슬루 · 단락전류 · 배선지연 · 리피터 · 도착시각 · 요구시각 · 슬랙 · OCV · NLDM · 비선형지연모델

기술자 넷에게 인버터의 지연이 얼마냐고 물으면 답이 넷 나오고, 넷 다 맞다. 서로 다른 네 모형으로 답하고 있기 때문이다. SPICE 를 쓰는 사람은 미분방정식으로 답한다. 물리 설계 기술자는 Liberty 파일의 2 차원 찾아보기 표로 답한다. 마이크로아키텍트는 FO4 로 답한다. 합성 기술자는 피코초가 하나도 안 들어간 무차원 수로 답한다. 이것들은 하나의 진실을 두고 겨루는 근사가 아니다. 서로 다른 일을 맡은 네 개의 연장이고, 엉뚱한 것을 쓰는 것이 바로 **실패한 경로와 무관했던 수**로 설계를 사인오프하게 되는 길이다.

이 장은 그 넷을 차례로, 앞의 것에서 뒤의 것을 지어 나가고, 걸음마다 그 모형이 옳은 연장이 되는 영역을 말한다. 여기 나오는 수는 전부 이 장의 소스 맨 위에 선언된 공정 상수 한 벌 — 0.9 V 의 28 nm 급 저전력 노드 — 에서 계산한 것이고, 아래의 모든 표는 손으로 타이핑한 것이 아니라 그것에서 계산된다.

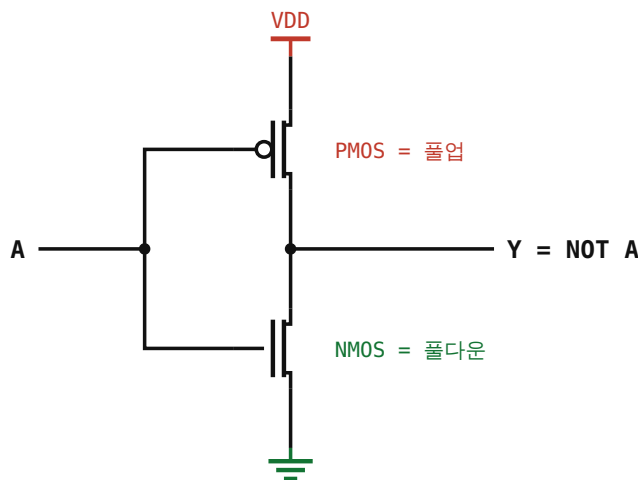
T2.1 머릿속으로 할 수 있는 단 하나의 모형: 스위치 달린 RC

CMOS 게이트는 저항처럼 동작하는 트랜지스터를 거쳐 축전기를 채우고 비운다. 그것이 전부다. 이 장의 나머지는 전부 저 한 문장을 다듬은 것이고, 다듬을 때마다 저 문장 안의 어떤 가정이 깨졌기 때문에 다듬은 것이다.

RC지연 (RC delay) — 게이트의 출력 마디는 등가 저항 R 을 거쳐 구동되는 용량 C 다. 그 마디의 전압은 감쇠하는 지수함수이고, 지연은 받는 쪽의 스위칭 문턱을 지나가는 데 걸리는 시간 — 관례로 V_{DD} 의 50 % — 이다.

유도 — 트랜지스터에서 0.69 RC 까지

	이렇게 된다	왜냐하면
1	스위칭하는 게이트는 출력 마디를 PMOS 쌍기를 거쳐 V_{DD} 에 잇거나 NMOS 쌍기를 거쳐 접지에 잇는다.	정적 CMOS 게이트가 바로 그것이다. 서로 보완하는 두 회로망이 있고, 정상상태에서는 정확히 하나만 도통한다.
2	스위칭하는 동안 도통하는 트랜지스터는 저항이 아니다 — 포화에서 시작해 선형으로 끝난다.	V_{DS} 가 V_{DD} 에서 시작해 0 으로 떨어지므로, 소자가 천이 중에 영역을 건너간다 (T1).
3	그래도 우리는 그것을 등가 저항 R_{eq} 하나로 같음한다. 같은 50 % 통과 시간을 주는 값으로 정의한다.	이것은 켄 저항이 아니라 맞춰 넣은 양이다 — 책마다 값이 다른 까닭이고, 출처가 다른 R_{eq} 를 서로 견주면 안 되는 까닭이다.
4	그러면 떨어지는 출력에 대해 $V(t) = V_{DD} \cdot e^{-t/RC}$ 다.	계단 입력을 받는 1 차 선형 상미분방정식의 표준해다.
5	$V = V_{DD}/2$ 로 놓으면 $t = RC \cdot \ln 2 = 0.69 RC$ 다.	$\ln 2 = 0.693$. 모든 지연 식에 나오는 0.69 가 여기서 온다 — 그것은 50 % 지점일 뿐 그 이상이 아니다.
6	10-90 % 상승/하강 시간은 같은 해에서 $t = RC \cdot \ln 9 = 2.2 RC$ 로 나온다.	같은 지수함수, 다른 두 통과점. 0.69 와 2.2 는 같은 파형을 말한다 — 지연과 슬루는 한 곡선을 두 번 읽은 것이다.



입력이 0 -> 위 (PMOS)가 켜짐 -> Y 를 VDD 로 당겨 올린다
 입력이 1 -> 아래 (NMOS)가 켜짐 -> Y 를 GND 로 당겨 내린다
 둘이 동시에 켜지지 않는다 -> 정상상태 전류가 0 이다

그림 25. 단위 인버터. 풀업과 풀다운 회로망이 RC 의 R 이고, 다음 단의 게이트 용량과 배선이 C 다.

$\tau = R_{eq} \cdot C_g$ — 이 공정의 시간 단위

기준 소자를 하나 고른다 — 최소 크기 인버터 — 그리고 수 둘을 준다: 등가 출력 저항 $R_{eq} = 8\text{ k}\Omega$ 과 입력 용량 $C_g = 1\text{ fF}$. 그 곱 $\tau = 8\text{ ps}$ 가 이 장의 나머지에서 유일한 절대 시간이다. 다른 모든 지연은 그것의 배수로 적힌다.

이것은 꾸밈이 아니다. 게이트 사이의 비는 위상이 정하고 공정·전압·온도가 바뀌어도 몇 퍼센트 안에서 안정적이다. 절대값 τ 는 코너를 건너가면 **2 배 넘게** 움직인다. 그 둘을 갈라 두어야 종이 위에서 아키텍처를 하고 피코초는 도구가 대게 할 수 있다.

어디에 쓰나

임계 경로를 손으로 어림하는 모든 자리, 이른 아키텍처 맞바꿈 전부, RTL 이 생기기 전에 '이게 한 주기에 들어가나' 를 묻는 모든 자리.

언제 쓰나

합성 전에 — STA 를 걸 넷리스트가 아직 없을 때. 그리고 STA 뒤에, 나온 수가 그럴듯하기라도 한지 확인할 때.

어떻게 쓰나

경로를 τ 로 적고 마지막에 한 번만 곱한다. 쓸 만한 기준점이 **FO4** 다 — 인버터가 자기 복사본 넷을 모는 것 — 이 공정에서는 40 ps 다. 요즘의 고성능 파이프라인 한 단은 대략 12-20 FO4 이고, 저전력이면 25-40 이다.

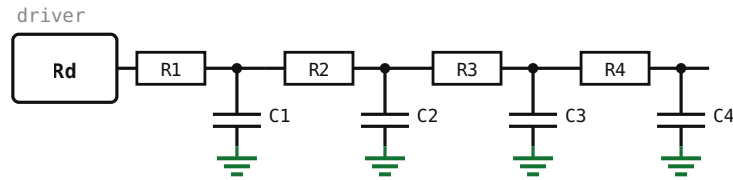
실제 코드는 이렇게 생겼다

```
// A 'timing shell' exists so that the number you measure is about
// your logic, not about the pads. Synthesise the DUT inside registers:
module fir_shell (input clk, input rst, input signed [7:0] din, output signed [15:0] dout);
    reg signed [7:0] din_q;
    reg signed [15:0] dout_q;
    wire signed [15:0] y;
    always @(posedge clk) begin din_q <= din; dout_q <= y; end
    fir4 u_dut (.clk(clk), .rst(rst), .x(din_q), .y(y));
    assign dout = dout_q;
endmodule
// Measured on this repository's own FIR: 455 MHz without the shell, 47 MHz with it.
// The 455 MHz was the tool timing a path that started and ended at a pin.
```

여기서 틀린다. 입력과 출력이 핀인 설계에서 F_{max} 를 인용하는 것. IP 데이터시트에서 가장 흔한 거짓 수치다. 경로에 띄우는 레지스터도 받는 레지스터도 없으면 도구는 제약할 것이 없어서 찾아낸 조합 지연을 그대로 찍는다 — 흔히 10 배 낙관적이다.

T2.2 엘모어 지연 — RC 하나가 사슬이 될 때

단일 RC 모형은 저항이 한쪽 끝에, 용량이 다른 쪽 끝에 있다고 본다. 배선은 그 둘을 다 어긴다. 저항도 용량도 배선을 따라 퍼져 있다. 그것을 N 토막으로 잘라 더한 것이 엘모어 지연이고, 그 합의 모양은 외워 둘 값어치가 있다. 긴 배선이 왜 비례해서 줄지 않는지의 까닭이 거기 있기 때문이다.



조각 i 의 저항은 뒤에 달린 용량 전부를 민다 -- 항이 $1+2+\dots+N$ 로 는다
 그래서 배선 지연은 길이에 비례하지 않고 길이의 제곱으로 크다

그림 26. 배선을 RC 네 토막으로 자른 것. 1 번 토막의 저항은 1-4 번 토막의 용량을 전부 밀어야 하고, 4 번 토막의 저항은 제 것만 민다.

유도 — 배선 지연이 왜 길이의 제곱으로 자라나

	이렇게 된다	왜냐하면
1	길이 L 인 배선을 같은 크기 N 토막으로 자른다. 토막마다 저항 R/N , 용량 C/N .	$R = r \cdot L$, $C = c \cdot L$ 이고, r 은 $\Omega/\mu\text{m}$, c 는 $\text{F}/\mu\text{m}$ 로 기술 파일에서 온다.
2	i 번 토막보다 뒤에 있는 모든 용량의 전하는 i 번 토막의 저항을 지나가야 한다.	전하가 거기 닿는 길이 하나뿐이다 — 엘모어 모형의 내용이 그것뿐이다.
3	그래서 지연은 $\sum_i (R/N) \cdot (N-i+1) (C/N) = RC \cdot (N+1)/(2N)$ 이다.	등차급수 $1+2+\dots+N = N(N+1)/2$. 대입하면 닫힌 꼴이 나온다.
4	$N \rightarrow \infty$ 이면 $RC/2$ 로 가고, R 과 C 는 각각 L 에 비례한다.	따라서 $t \propto r \cdot c \cdot L^2$ — 배선 길이를 두 배로 하면 지연이 네 배가 된다. 게이트 지연은 전혀 안 변하는데도 그렇다.
5	이 공정에서 $1, \mu\text{m}$ 배선은 $RC/2 = 90 \text{ ps}$ 를 주고, 네 토막 근사는 112 ps 를 준다.	거친 토막내기는 1.25 배 과대평가한다. 퍼져 있는 경우에는 닫힌 꼴을 쓰고, 토막은 그 토막이 실제로 있을 때만(비아, 가지, 탭) 쓴다.

표 70. 퍼진 배선을 토막내기: 몇 토막이면 되나

토막 수 N	엘모어 지연	퍼진 $RC/2$ 에 대한 오차
1	18 ps	1 %
2	135 ps	50 %
4	112 ps	25 %
8	101 ps	12.5 %
16	95.6 ps	6.25 %
32	92.8 ps	3.12 %

저 표에 사실 둘이 숨어 있다. 첫째, **한 토막은 100 % 틀린다** — 그리고 한 토막이야말로 순진한 스프레드시트가 하는 짓이다. 둘째, 수렴이 느리다. 8 토막이어도 여전히 6 % 높다. 추출 도구가 아예 토막을 안 내고 퍼진 회로망을 푸는 까닭이 이것이다. 손 여림은 $RC/2$ 를 쓰되, 그것이 추출기가 낼 값의 하한이라는 것을 기억하라. 추출기는 이웃과의 결합 용량까지 보기 때문이다.

엘모어 지연

RC 나무의 임펄스 응답의 1 차 모멘트다. 마디 k 에 대해 $t_k = \sum_i R_{ik} \cdot C_i$ 이고, R_{ik} 는 마디 i 로 가는 길과 마디 k 로 가는 길이 함께 쓰는 저항이다. 단일 RC 에서는 정확하고, 계단 입력을 받는 RC 나무의 50 % 지연에 대해서는 늘 **상한**이며, 인덕턴스는 전혀 안 본다.

어디에 쓰나	클럭 트리 합성(요의 균형 맞추기), 버퍼와 리피터 끼우기, 그리고 모든 배치기의 증분 타이밍 엔진 안에 든 지연 모형 — 배치는 한 번 옮길 때마다 전체 추출을 할 여유가 없다.
언제 쓰나	나무 구조에 대해 단조롭고 값싸게 갱신되는 지연이 필요할 때마다. 절댓값으로는 10-30 % 틀리고 순서로는 맞는데, 최적화기가 필요한 것이 바로 그 순서다.
어떻게 쓰나	구동점에서 나무를 따라 내려가며 앞마다 함께 쓰는 저항을 누적한다. 절대로 사인오프에 쓰지 마라 — 이것은 최적화 목적함수이지 사인오프 모형이 아니다.

실제 코드는 이렇게 생겼다

```
# Extraction produces this; you read it when a net is suspect.
# SPEF, standard parasitic exchange format, one net:
*D_NET net_dout_3 1.84e-01          $ total capacitance, pF
*CONN
*I u_fir/Z 0 *L 0 *D INVX4          $ driver
*I u_reg/D I *L 1.02e-03            $ receiver pin cap, pF
*CAP
1 net_dout_3:1 2.31e-02
2 net_dout_3:2 net_clk:7 4.10e-03    $ COUPLING to an aggressor -- not in your hand estimate
*RES
1 net_dout_3:1 net_dout_3:2 1.79e+01 $ ohms
```

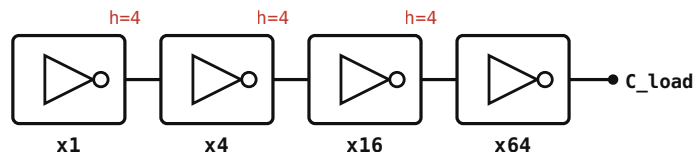
여기서 틀린다. 엘모어는 계단 입력에 대한 상한이다. 진짜 입력은 슬루가 유한하고, 빠른 입력이 긴 배선으로 들어가면 실제 50 % 지연이 그 상한을 넘을 수 있다. 슬루가 크면 당신이 기대는 그 상한은 배운 그 상한이 아니다 — 그것이 다 음절이다.

T2.3 논리적 노력 — 피코초를 덜어 낸 지연

게이트 사슬의 치수를 RC 셈으로 정하는 것은 되기는 되는데 괴롭다. 논리적 노력은 문제에서 공정을 통째로 걷어내고, 머릿속에서 최적화할 수 있는 무차원 식만 남긴다. 그리고 실제로 부딪히는 두 물음에 답한다: 단을 몇 개로 할 것인가, 그리고 단마다 얼마나 커져야 하는가.

유도 — 한 단의 정규화된 지연

	이렇게 된다	왜냐하면
1	게이트의 지연을 $d = (R_{\text{gate}} \cdot C_{\text{out}}) + (R_{\text{gate}} \cdot C_{\text{self}})$ 로 적는다.	출력 마디는 부하 와 그 게이트 자신의 드레인 용량을 같이 진다. 둘 다 같은 저항을 거쳐 채워진다.
2	전체를 $\tau = R_{\text{inv}} C_g$ 로 나눈다.	모든 것을 최소 인버터에 견주어 재기로 고른 것이다. 잃는 것은 없다. 단위만 없어진다.
3	첫 항이 $g \cdot h$ 가 된다. 논리적 노력 g 는 이 위상이 전류를 모는데 인버터보다 얼마나 나쁜가이고, 전기적 노력 $h = C_{\text{out}} / C_{\text{in}}$ 이다.	g 는 오직 위상에만 달려 있고(NAND2 = 4/3, NOR2 = 5/3, 인버터 = 1), h 는 부하 대 입력의 비에만 달려 있으므로 절대 크기와 무관하다.
4	둘째 항이 p , 기생 지연 이 된다 — 아무것도 안 모는 게이트의 지연.	그 게이트 자신의 확산과 함께 자라는데, 확산은 구 동력이 자라는 것과 똑같이 폭을 따라 자란다 — 그래서 p 는 크기와 무관하다. $p_{\text{inv}} \approx 1$.
5	$d = g \cdot h + p$.	곱 두 번과 덧셈 한 번, 그리고 공정 상수는 어디에도 없다.
6	N 단 경로에서 합은 모든 단이 같은 단 노력 $f = (G \cdot B \cdot H)^{1/N}$ 을 질 때 최소가 된다.	단마다의 크기로 합을 미분해 0 으로 놓으면 대칭해가 떨어진다. $G = \prod g$, $H = C_{\text{load}} / C_{\text{in}}$, B 는 경로에서 갈라져 나가는 가지를 센다.



한 칸의 지연 = $g \cdot h + p$. 사실 전체는 칸마다 같은 h 일 때 가장 빠르다
 그 h 가 대략 **3.6** (전기적 노력의 N 제곱근) -- **4** 로 잡고 시작한다

그림 27. 네 단, 단마다 앞의 4 배. 합을 최소로 만드는 것은 단 노력이 같은 것이지, 크기가 같은 것도 마지막 단 하나가 거대한 것도 아니다.

예제 11. 64 배 부하를 몬다: 단을 몇 개로?

주어진 것. 1 배 인버터가 $64 \cdot C_g$ 부하(받는 쪽이 여럿인 칩 내부 버스)를 몰아야 한다. 인버터 사슬의 논리적 노력은 $G = 1$, 갈래가 없으므로 $B = 1$, $H = 64$ 이고 경로 노력 $F = 64$ 다.

푸는 법. N 마다 $D = N \cdot F^{1/N} + N \cdot p_{inv}$ 를 계산해 가장 작은 것을 고른다. 아래 표는 외운 것이 아니라 계산한 것이다.

답. $N = 3$ 단, 단 노력 $f = 4$, 총 지연 $15 \cdot \tau = 12$ ps. 한 단으로 하면 52 ps 가 걸린다 — **4.33 배 느리다.**

표 71. $F = 64$ 일 때 단 수에 따른 총 지연

단 수 N	단 노력 f	지연 (τ)	지연
1	64	65	52 ps
2	8	18	144 ps
3	4	15	12 ps
4	2.83	15.31	123 ps
5	2.3	16.49	132 ps
6	2	18	144 ps

여기서 틀린다. 표의 최솟값을 날카로운 최적점으로 읽는 것. $N = 3$ 과 $N = 4$ 는 2.1 % 차이다 — 어떤 실제 p 의 값에 안에 들어가는 차이다. 이름뿐인 빠른 쪽이 아니라 싼 쪽을 고르라. 단 하나를 더하면 면적과 누설과, 그리고 그다음에 몰아야 할 슬루가 따라온다.

표 72. 실제로 인스턴스화하게 될 셀들의 논리적 노력

게이트	g	p (τ)	g 가 왜 그 값인가
인버터	1	1	기준
NAND2	4/3	2	NMOS 둘이 직렬: 같은 풀다운 전류를 내려면 각각 2 배 넓어야 하므로 입력 용량이 입력당 4/3 로 오른다
NAND3	5/3	3	셋이 직렬 — g 는 쌓기 깊이에 선형으로 자란다
NOR2	5/3	2	PMOS 가 직렬인데 PMOS 는 약한 소자다. 빠른 논리에서 NOR 를 피하고 NAND 는 안 피하 는 까닭이 이것이다
NOR3	7/3	3	같은 까닭, 더 나쁘다
XOR2	4	4	두 입력의 두 극성이 다 내부에 필요하다
MUX2	2	4	전달 경로 둘에 선택 구동까지
삼상태 인버터	2	2	인에이블 소자가 출력과 직렬로 붙는다 — 전형적인 버스 드라이버

설계 연장으로 쓰는 논리적 노력

$d = g \cdot h + p$. 경로 전체는 $D = \sum(g_i h_i) + \sum p_i$ 이고, 단 노력이 $f = F^{1/N}$ 으로 같을 때 최소다. 실무 규칙은 $f \approx 4$ ($p_{inv} = 1$ 일 때의 참 최적은 3.6)이고, 단을 하나 더하는 것은 그것이 p 보다 많은 지연을 견어 낼 때만 값어치가 있다.

어디에 쓰나	합성 전에 파이프라인 한 단을 얼마나 깊게 할 수 있는지 정할 때. NAND-NAND 로 할지 NOR-NOR 로 할지 고를 때. 클럭이나 리셋 분배를 손으로 치수 잡을 때. 이상해 보이는 합성 결과와 다룰 때.
언제 쓰나	아키텍처 단계에서, 그리고 도구의 답이 온전한지 확인해야 할 때마다. 합성은 속으로 이것을 하지만 면적과 전력까지 저울에 다는 비용함수를 쓰므로 답이 다르게 나온다 — 이상값을 알아야 그 차이가 맞바꿈인지 실수인지 가를 수 있다.
어떻게 쓰나	경로를 따라 G 를 세고, 실제 부하에서 H 를 내고, $F = G \cdot B \cdot H$ 를 잡는다. 그러면 $N \approx \log_4 F$ 이고 $f = F^{1/N}$ 이다. 부하에서 거꾸로 올라가며 단마다 치수를 낸다: $C_{in,i} = g_i \cdot C_{out,i} / f$.

실제 코드는 이렇게 생겼다

```
# The synthesis constraint that makes the tool solve the same problem.
# Without a real load and a real driver, the tool optimises a fiction.
set_load      0.064      [get_ports dout*]    ;# pF -- from the receiver's .lib
set_driving_cell -lib_cell INVX4 [get_ports din*]    ;# not an ideal source
set_max_transition 0.150 [current_design]          ;# ns -- bounds slew degradation
set_max_fanout 16      [current_design]
# And the one that is almost always missing on a first pass:
set_max_capacitance 0.200 [current_design]
```

여기서 틀린다. g 는 위상의 성질이지 라이브러리에서 고른 셀의 성질이 아니다. 라이브러리의 NAND2_X4 와 NAND2_X1 은 g 가 같다. 다른 것은 C_{in} 뿐이다. 단 수를 안 바꾸고 더 큰 셀로 갈아 끼워 '타이밍을 고쳤다' 고 하는 기술자는 앞 단의 h 를 바꾼 것이다 — 문제를 상류로 옮긴 것이고, 사슬이 길면 그것이 돌아온다.

T2.4 슬루, 그리고 지연에 인자가 둘인 까닭

지금까지의 두 모형은 입력을 하나만 받는다: 부하. 실제 타이밍 도구는 둘을 받는다. 게이트의 지연이 그 입력이 얼마나 빨리 움직였는지에 달려 있기 때문이다. 느린 입력은 두 트랜지스터를 어정쩡하게 켜 둔 채로 두어 쌓기를 통해 전류를 버리고, 출력을 늦게 띄운다. 그것이 위의 모든 것에 대한 가장 큰 보정 하나이고, Liberty 파일이 식이 아니라 표를 들고 있는 까닭이다.

슬루 (slew / transition time) — 신호가 V_{DD} 의 정해진 두 비율 사이를 지나는 데 걸리는 시간. 관례로 10 %-90 % 또는 30 %-70 % 다. 어느 쌍인지는 라이브러리의 성질이고, 서로 다른 쌍을 쓰는 라이브러리를 섞으면 설계의 모든 지연이 조용히 바뀐다.

단락전류 (short-circuit current) — 천이하는 동안 풀업과 풀다운 회로망이 동시에 도통해서 쌓기를 통해 V_{DD} 를 접지에 단락시킨다. 이렇게 잃는 전하는 입력 천이 시간에 비례한다. 보통의 슬루에서는 스위칭 에너지의 5-15 % 이고, 슬루가 망가지면 쓸모 있는 스위칭 에너지를 넘을 수도 있다.

표 73. 같은 인버터, 서로 다른 '지연' 넷 — NLDM 의 한 조각

입력 슬루 (ps)	부하 4 fF	부하 16 fF	부하 64 fF
20	16 ps	40.4 ps	137 ps
60	24 ps	49.2 ps	147 ps
120	36 ps	62.4 ps	162 ps
240	60 ps	88.8 ps	192 ps

저 표는 모형이지 켄 것이 아니다 — 슬루 계수는 보이기 위한 값이다. **그렇다고 말하라.** 진짜 NLDM 표는 코너 전체에 걸쳐 격자점마다 SPICE 를 돌려 특성화한 것이고, 보통 이음마다 · 셀마다 · 코너마다 7×7 점이다. 이 표에서 진짜인 것은 모양이다. 지연은 부하에 대체로 선형으로, 입력 슬루에 대체로 선형으로 오르고, **슬루 항은 작지 않다** — 가벼운 부하에서는 그것이 지배 항이다.

NLDM — 비선형 지연 모형, 그리고 도구가 실제로 읽는 것

Liberty (.lib) 파일은 셀마다, 타이밍 이음마다 2 차원 표 둘을 준다 — 하나는 지연, 하나는 출력 슬루 — 그리고 입력 천이 시간과 총 출력 용량으로 색인한다. 도구는 쌍선형으로 보간해서(가장자리에서는 **외삽해서**) 수를 낸다. 그 뒤의 모든 것 — STA, 합성 비용, 배치 타이밍 — 이 이 표 위에 서 있다.

그 후계인 CCS(composite current source)는 스칼라가 아니라 전류 파형을 저장한다. 받는 쪽의 입력 용량 자체가 전압에 따라 변하거나, 넷의 저항이 커서 드라이버와 넷이 서로 간섭할 때 그 차이가 문제가 된다. IP 납품물에는 **당신의 수가 어느 모형에서 나왔는지 적으라**. 5 % 수준에서 둘은 바꿔 쓸 수 있는 것이 아니다.

어디에 쓰나	합성 · STA · 배치배선의 모든 지연 수치. 그리고 간접적으로 IP 데이터시트에도: 당신의 $F_{\max}$ 는 라이브러리와 코너를 같이 적어야 뜻이 생긴다.
언제 쓰나	.lib 를 직접 읽게 되는 경우는 딱 둘이다. 어떤 경로의 지연이 말이 안 될 때 도구가 표 밖으로 외삽했는지 보려고, 그리고 한 번도 본 적 없는 라이브러리를 지원하라고 할 때.
어떻게 쓰나	이름을 찾아 index_1(입력 슬루)과 index_2(부하)를 읽고, 당신의 동작점이 둘 다의 안쪽인지 확인한다. 표 밖 외삽이 '도구는 통과라는데 실리콘은 아니다' 의 표준적인 까닭이다.

실제 코드는 이렇게 생겼다

```
/* Liberty, one arc of one cell -- this is what the tool reads. */
cell (INVX1) {
  area : 1.596 ;
  pin (A) { direction : input ; capacitance : 1.02 ; }      /* fF */
  pin (Z) {
    direction : output ;
    function : "!A" ;
    max_capacitance : 64.0 ;
    timing () {
      related_pin : "A" ;
      timing_sense : negative_unate ;
      cell_fall (delay_7x7) {
        index_1 ("0.004, 0.012, 0.028, 0.060, 0.124, 0.252, 0.508"); /* input slew, ns */
        index_2 ("0.4, 1.6, 4.0, 8.8, 18.4, 37.6, 76.0");           /* load, fF */
        values ( "0.0091, 0.0142, 0.0241, 0.0431, 0.0812, 0.1574, 0.3098", \
                  "0.0113, 0.0164, 0.0263, 0.0453, 0.0834, 0.1596, 0.3120", \
                  /* ... five more rows, one per input slew ... */ );
      }
      fall_transition (delay_7x7) { /* the OUTPUT slew -- this feeds the next stage */ }
    }
  }
}
```

여기서 틀린다. 지연과 출력 슬루는 서로 다른 표 둘이고, 출력 슬루는 다음 단의 입력 슬루다. 잘못 몰린 넷 하나가 경로 전체를 이렇게 오염시킨다. 단마다 더 나쁜 슬루를 받아 더 나쁜 슬루를 내놓고, 그 악화가 누적된다.

set_max_transition 이 있는 까닭이 그 누적을 막는 것이고, max_transition 제약이 없는 설계는 보고서에서는 셋업을 맞추고 실리콘에서는 실패할 수 있다.

T2.5 배선, 그리고 그것을 구해 주는 리피터

게이트 지연은 공정 노드마다 줄어든다. 배선 지연은 안 줄어든다. 배선이 가늘어지면서 r 은 오르고, 마이크로미터 당 c 는 대체로 그대로이며, 다이는 트랜지스터만큼 빨리 안 줄어든다. 노드마다 짧아져 온 어떤 길이를 넘으면 배선이 경로를 지배하고, 해답은 더 큰 드라이버가 아니다 — 배선이 길지 않게 만드는 것이다.

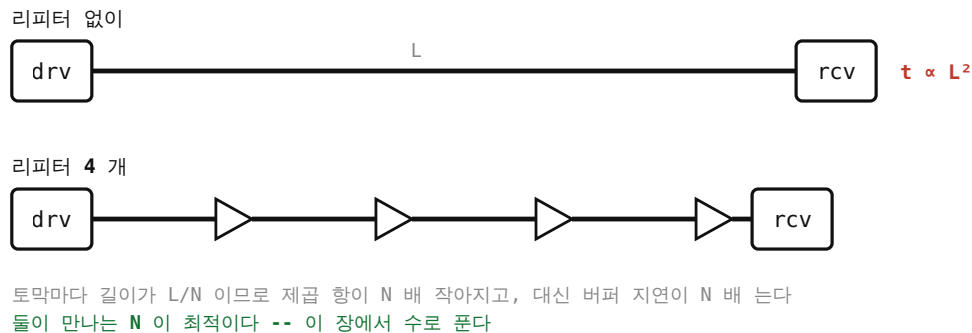


그림 28. 긴 배선, 그리고 리피터로 토막 낸 같은 배선. 제곱 항은 N 분의 1 로 줄고 버퍼 지연은 N 배로 늘어난다.

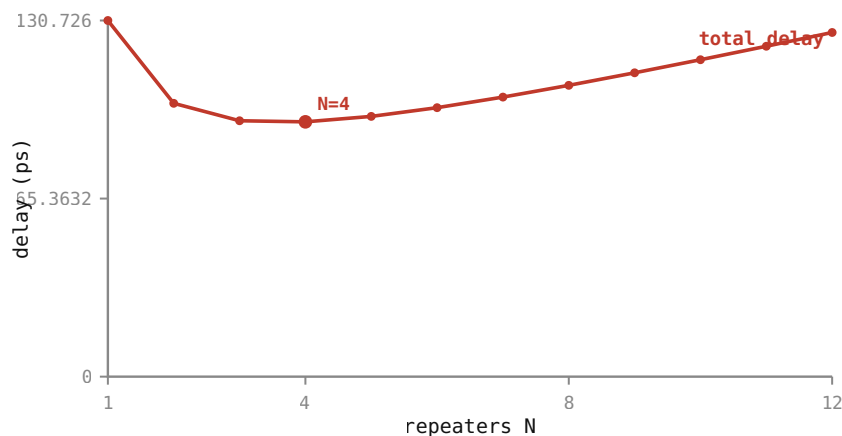


그림 29. 1, μm 배선에 대한 전체 쓰기. 점마다 그 점의 최적 리피터 크기를 썼다. **최솟값이 같다** — $N = 3, 4, 5$ 가 서로 2.1 % 안에 있다 — 그러니 배선이 되는 쪽을 고르라.

유도 — 최적의 어디서 나오나, 그리고 그것을 대조한다

	이렇게 된다	왜냐하면
1	배선을 k 토막으로 자르고 토막마다 크기 h 인 인버터로 만든다.	토막마다 리피터 하나. 마지막 것이 받는 쪽을 본다.
2	토막마다 드라이버가 $0.7 \cdot (R_{\text{inv}}/h)(C_W/k + h \cdot C_g)$, 퍼진 배선이 $0.4 \cdot R_S C_S$, 먼 쪽 부하가 $0.7 \cdot R_S \cdot h C_g$ 를 치른다.	0.4 와 0.7 은 표준적인 분포 RC 계수다 — 자기 용량을 모는 배선에는 0.4, 뭉친 부하에는 0.69.
3	k 와 h 를 수치로 쓰면 $k = 4$, $h = 38$ 에서 93.5 ps 가 나온다.	격자 전체를 무시하게 뒀던 것이다 — 최적의 모양에 대해 아무것도 가정하지 않았다.
4	달힌 꼴 $k = \sqrt{0.4 R_W C_W / 0.7 R_{\text{inv}} C_g}$ 과 $h = \sqrt{R_{\text{inv}} C_W / R_W C_g}$ 은 $k = 3.59$, $h = 37.9$ 를 준다.	독립 대조다. 같은 답으로 가는 두 길이 크기에서 0.1 % 안에서 맞는다. 아무도 대조하지 않은 유도 하나가 틀린 상수를 몇 해씩 살려 두는 길이다.
5	리피터 없이 같은 배선을 그 배선의 최적 드라이버로 물면 131 ps 다.	그러니 여기서 리피터가 사 주는 것은 1.4 배다 — 그리고 그것이 정직한 표제다. 쓰기에서 가장 유리한 구석에서 집어 온 수가 아니다.

리피터 끼우기

지연이 길이의 제곱인 배선을, 제곱 항이 버퍼 지연과 견줄 만해질 만큼 짧은 토막으로 끊는다. 최적 토막 길이는 **토막마다의 배선 RC가 버퍼 자신의 지연과 같아지는** 길이다. 그것을 넘으면 아무것도 못 아끼면서 버퍼 값만 치르는 것이다.

이 공정에서 갈림점은 배선 25 μm 언저리다 — 그보다 짧으면 리피터가 아끼는 것보다 더 든다.

어디에 쓰나	블록을 벗어나는 모든 넷: 블록 사이 데이터패스, 클럭 척추, 리셋 분배, 그리고 당신의 IP 포트가 닿는 SoC 최상위 조립.
언제 쓰나	물리 설계에서 정해지지만 당신의 납품물에서 제약된다 : 당신 IP의 출력에 200 ps 예산이 있는데 통합하는 쪽의 배선이 300을 요구하면, RTL에는 아무 잘못이 없는데 블록이 실패한다.
어떻게 쓰나	통합하는 쪽에 바람이 아니라 수를 주라. 출력 지연, 구동 세기, 최대 부하를 데이터시트에 적고 양쪽 끝을 <code>set_output_delay / set_load</code> 로 제약한다. 그러면 리피터 문제는 그쪽 것이 되고, 풀 수 있는 문제가 된다.

실제 코드는 이렇게 생겼다

```
# What the integrator must be told, and what you must constrain.
# In your IP's SDC (shipped with the deliverable):
create_clock -name clk -period 4.000 [get_ports clk]
set_output_delay -max 1.200 -clock clk [get_ports {dout[*] dvalid}]
set_input_delay -max 1.000 -clock clk [get_ports {din[*] dvalid_i}]
set_load 0.064 [get_ports dout[*]] ;# what we assumed -- say it in the datasheet
# And in DATASHEET.md, the line that prevents the support call:
# Output timing assumes <= 64 fF load and <= 150 ps transition at the port.
# Beyond that, insert a repeater; every 100 fF adds approximately 25 ps.
```

여기서 틀린다. 리피터는 반전한다. 홀수 개면 넷의 극성이 뒤집히고, 그 해결책 — 인버터 대신 버퍼(인버터 둘)를 쓰는 것 — 은 인버터 사슬보다 지연이 더 든다. 도구가 이것을 처리하지만, **리셋이나 스캔 인에이블 경로에 손으로 놓은 리피터는 전형적인 브링업 버그다**: 한 코너에서만 블록이 틀린 극성으로 리셋된다. 끼워 넣은 한 쌍이 바로 그 코너에서 최적으로 지워졌기 때문이다.

T2.6 셀에서 경로로: STA가 실제로 더하는 것

정적 타이밍 분석은 시뮬레이션하지 않는다. 그래프를 따라 수 둘을 민다 — 신호가 도착할 수 있는 가장 이른 시각과 가장 늦은 시각 — 그리고 끝점마다 요구하는 것과 견준다. 위의 모든 것이 그 그래프의 이름 하나를 댄다. 이 절은 그 이름들이 어떻게 판정이 되는가다.

유도 — 셋업과 홀드를 각각 부등식 하나로

	이렇게 된다	왜냐하면
1	띄우는 플롭이 $t_{\text{launch}} = T_{\text{clk,launch}} + t_{\text{cq}}$ 에 데이터를 내놓는다.	클럭은 띄우는 플롭에 그 플롭 나옴의 시각에 닿는다 — 클럭 지연은 경로의 양 끝에서 같지 않다.
2	그것이 받는 플롭에 $t_{\text{arrival}} = t_{\text{launch}} + t_{\text{logic}}$ 에 닿는다.	t_{logic} 이 이 장이 지금까지 계산해 온 전부다.
3	받는 예지는 $T_{\text{clk,capture}} + T_{\text{period}}$ 에 오고, 데이터는 그보다 t_{setup} 앞서 안정돼 있어야 한다.	그 안정 요구는 플롭의 성질이고, 다른 이음과 똑같은 방식으로 특성화된다.
4	셋업: $t_{\text{arrival}} \leq T_{\text{capture}} + T - t_{\text{setup}} - t_{\text{uncertainty}}$ 슬랙은 그 차다.	클럭 불확실도가 지터와, (CTS 전이라면) 도구가 아직 알 수 없는 스큐를 흡수한다. 슬랙이 양이면 통과, 음이면 실패다.
5	홀드: 가장 빠른 경로가 같은 받는 예지의 홀드 창보다 먼저 닿으면 안 된다: $t_{\text{arrival,min}} \geq T_{\text{capture}} + t_{\text{hold}}$	홀드에는 T가 없다. 클럭을 느리게 해도 홀드 위반은 안 고쳐진다 — 처음 설계하는 사람에게 이 장에서 가장 쓸모 있는 사실 하나다.

표 74. 경로의 수가 당신 어림보다 나빠지는 네 가지 길

효과	무엇을 하나	대략 크기	어디서 정하나
OCV / AOCV / POCV	띄우는 클럭 경로와 받는 클럭 경로의 셀·넷 지연을 서로 다르게 디레이트한다. 같은 다이 위의 똑같은 셀 둘이 똑같지 않기 때문이다	클럭 경로의 3-10 %	set_timing_derate
클럭 불확실도	지터를, 그리고 CTS 전이면 스큐까지 대신하는 평평한 뽀렘	50-150 ps	set_clock_uncertainty
CPPR / CRPR	띄우는 클럭 경로와 받는 클럭 경로가 함께 쓰는 부분에 두 번 걸린 디레이트를 돌려준다	10-50 ps 를 되찾는다	사인오프 STA 에서는 기본으로 꺼짐. 합성에서는 흔히 꺼짐
SI / 누화 지연	반대 방향으로 스위칭하는 가해자가 당신의 천이를 늘린다	길게 나란한 넷에서 5-15 %	SPEF 의 결합을 넣은 SI 인지 STA

report_timing 을 속지 않고 읽기

타이밍 보고서는 장부다. 띄우는 플롭까지의 클럭 경로, 데이터 경로를 따르는 셀과 넷의 지연, 받는 플롭까지의 클럭 경로, 라이브러리 셋업, 그리고 디레이트. **그 보고서가 뜻이 있는지를 정하는 줄은 그것이 어느 코너와 어느 모드에서 돌았는가 하나다** — 이상 클럭으로 전형 코너에서 나온 보고서는 아무것에 대한 증거도 아니다.

어디에 쓰나	모든 타이밍 사인오프, 모든 '이게 왜 실패하지' 조사, 그리고 데이터시트에 적는 모든 F_{max} 수치.
언제 쓰나	슬랙이 음이면 경로 전체를 읽는다. 양이면 시작점과 끝점을 먼저 읽어서, 도구가 당신 생각의 그 경로를 켜진 것이 맞는지 확인한다. 거짓 경로와 제약 없는 끝점은 여기 말고 다른 데서는 안 드러난다.
어떻게 쓰나	확인할 것: (1) 끝점이 포트가 아니라 레지스터인가, (2) 클럭이 진짜(전파된 것)인가 이상 클럭인가, (3) 가장 나쁜 셀에서 입력 천이가 얼마인가 — .lib 색인 범위의 끝이면 그 수는 외삽된 것이다, (4) 경로에서 넷 지연이 얼마인가. 넷 지연이 40 % 를 넘으면 고칠 것은 논리가 아니라 배치다.

실제 코드는 이렇게 생겼다

```
# Startpoint: u_fir/acc_reg[7] (rising edge-triggered flip-flop clocked by clk)
# Endpoint:   u_fir/y_reg[15] (rising edge-triggered flip-flop clocked by clk)
# Corner: ss_0p81v_125c    Check: setup    Path Group: clk
# Point      Incr      Path
# clock clk (rise edge)      0.000      0.000
# clock network delay (propagated) 0.412      0.412    <- real CTS, not ideal
# u_fir/acc_reg[7]/CK (DFFX1) 0.000      0.412 r
# u_fir/acc_reg[7]/Q (DFFX1)  0.138      0.550 f
# u_fir/U231/Z (NAND2X2)      0.061      0.611 r
# u_fir/U248/Z (INVX4)        0.029      0.640 f
# net (fanout=6, length=214um) 0.087      0.727    <- 12% of the path is WIRE
# u_fir/y_reg[15]/D (DFFX1)   0.000      0.727 f
# data arrival time           0.727
# clock clk (rise edge)      4.000      4.000
# clock network delay (propagated) 0.438      4.438
# clock reconvergence pessimism 0.021      4.459    <- CPPR gives this back
# library setup time         -0.096      4.363
# data required time          4.363
# slack (MET)                 3.636
```

여기서 틀린다. 보고서에 clock network delay (ideal) 이라고 적혀 있고 (propagated) 가 아니라면, 클럭 트리가 아직 없다는 뜻이고 스큐는 짐작이다. 이상 클럭 보고서의 수는 사인오프에서 흔히 200-400 ps 움직인다 — 나쁜 쪽으로. 이상 클럭 F_{max} 를 절대 데이터시트에 적지 마라.

T2.7 월요일에 무엇을 할 것인가

표 75. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
RC지연 · τ · FO4	RTL 이 생기기 전의 모든 블록	목표 주파수에서 그 기능이 한 주기에 들어가나
엘모어지연	클럭 트리, 리셋 트리, 긴 데이터패스 넷	버퍼를 어디에 놓아야 하나, 어느 길이 늦은 것인가
논리적노력	임계 경로 재구성, 파이프라인 깊이	단 수, 그리고 단 사이의 크기 비
슬루 · NLDM	.lib 읽기, 말 안 되는 지연 디버깅	도구가 당신 동작점을 보간했나 외삽했나
배선지연 · 리피터	블록 경계와 최상위 조립	데이터시트에 내놓는 출력 부하와 천이 한계
도착시각 · 요구시각 · 슬랙	모든 사인오프	지금 팔려는 그 수가 진짜인가

실제로 난 일

이 저장소는 자기 FIR 필터를 **455 MHz** 로 켜다가 다시 **47.4 MHz** 로 켜다 — 9.6 배 차이 — 같은 RTL 에서. 다른 것은 타이밍 꺾이기 하나였다. 경계에 레지스터가 없으면 도구는 핀에서 핀까지 가는 경로의 지연을 찍는데, 그것은 칩 안에 존재하지 않는 경로다. 이 장의 모든 수는 그것을 켜 경로만큼만 뜻이 있고, 10 배 틀리는 가장 값싼 방법은 엉뚱한 경로를 재는 것이다.

다음 장은 이 장이 다룰 수 없는 딱 하나의 경우를 말한다. 셋업 부등식이 깨지는 까닭이 경로가 느려서가 아니라 데이터가 애초에 그 클럭과 동기가 아니었기 때문일 때 무슨 일이 일어나는가. 답은 더 긴 지연이 아니라 확률이고, 이 책에서 결코 0 으로 만들 수 없는 유일한 타이밍 양이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

게이트의 지연은 물리이고 특허가 안 된다. **지연이 문제가 되지 않도록 블록이 스스로를 배치하는 방식은 특허가 된다**. 실제로 팔리는 IP 의 청구항이 무엇을 덮는지 보라. 되먹임 굵셈-덧셈을 임계 고리에서 빼내 미리 계산한 다중화기로 갈음하는 투기적(언롤링) 결정 되먹임 등화기 — 산술은 그대로인데 고리 지연은 그대로가 아니다. 두 자리올림을 모두 계산해서 면적으로 지연을 사는 캐리 셀렉트 가산기. 느린 경로가 빠른 경로에서 시간을 빌리도록 두 띄우기 지점을 일부러 어긋나게 하는 클럭 트리 방식. 어느 경우든 발명은 지연을 어디로 옮겼는가 이지 트랜지스터가 얼마나 빠른가가 아니다. 이 장이 식만큼이나 어느 향이 지배하는가 에 시간을 쓰는 까닭이 그것이다: 짚어 내지 못한 향은 옮길 수도 없다.

T3. 준안정 — 설계로 없앨 수 없는 단 하나의 실패

다른 모든 타이밍 양은 통과하게 만들 수 있다. 이것만은 드물게 만들 수 있을 뿐이고, 얼마나 드문지를 당신이 말해야 한다.

읽기 전에 알아야 하는 것

전압 · 전류 · 미분 · 지수함수 · 로그 · 확률 · 평균 · 트랜스 컨덕턴스 · 출력저항 · 게이트용량 · RC지연 · 도착시각 · 요구시각 · 슬랙 · FO4

이 장에서 새로 나오는 것

준안정 · 결정시간상수 · 해결시간 · 준안정창 · MTBF · 동기화기깊이 · 다중비트문제 · 재수렴 · 그레이코드 · 핸드셰이크 · 비동기FIFO · 리셋동기화기 · 리커버리 · 리무벌 · CDC검증 · 준정적신호

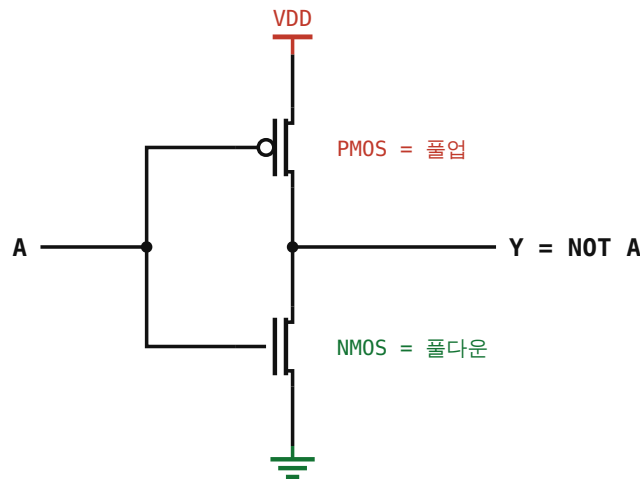
셋업 위반은 일정 문제다. 데이터가 늦었고, 클럭을 늦추거나 논리를 줄이면 고쳐진다. 준안정은 그것이 아니다. 플립플롭이 클럭 에지 바로 그 순간에 변하고 있는 입력을 표본화하면, 출력이 두 합법적인 준위 사이에 무한정 머물 수 있다. 아무리 느리게 해도 없어지지 않는다 — 표본화 순간과 데이터 천이는 서로 무관하므로 그 나쁜 우연은 또 일어난다. **물음은 얼마나 자주냐, 그리고 그것을 예산에 넣었느냐뿐이다.**

이 장은 그 물음에 수로 답한 다음, 나머지 분량을 더 흔한 실패에 쓴다. 수를 바르게 계산하고 2 단 동기화를 넣 고도 멈추는 칩을 내보내는 기술자들 — 경계를 건넌 것이 한 비트가 아니었기 때문이다.

T3.1 결정이 왜 무한정 걸릴 수 있나

유도 — 교차 결합 쌍에서 지수함수까지

	이렇게 된다	왜냐하면
1	플립플롭의 저장 소자는 고리를 이룬 인버터 둘이다. 각각의 출력이 다른 쪽의 입력을 몬다.	그 고리가 클럭 없이 상태를 붙드는 것이다 — 축전기를 따로 이름 붙이지 않고도 기억을 갖는 디지털 설계의 유일한 구조다.
2	그 고리에는 평형점이 셋 있다. 단단한 0, 단단한 1, 그리고 두 인버터가 각자의 스위칭 문턱에 앉는 가운데 한 점.	V_{out1} 대 V_{in1} 과 두 번째 인버터의 거울 곡선을 그리면 세 번 만난다. 가운데 교점이 준안정점이다.
3	가운데 평형은 불안정 하다. 거기서 고리 이득이 $A^2 > 1$ 이므로 어떤 벗어남이든 자란다.	인버터마다 문턱에서 소신호 이득 $A = -g_m(r_{o,n} \parallel r_{o,p})$ 를 갖고, T1 이 두 항을 다 줬다.
4	그 둘레에서 선형화하면 $C \cdot dv/dt = (A - 1) \cdot v/R$, 즉 $v(t) = v_0 \cdot e^{t/\tau}$ 이고 $\tau = RC/(A - 1)$ 이다.	여기서 τ , 결정 시간상수가 나온다 . 그것은 회로의 성질이고 — 대략 고리의 RC 를 그 남는 이득으로 나눈 것 — 당신이 고를 수 있는 규격이 아니다.
5	올바른 준위로 결판내려면 고리가 벗어남을 v_0 에서 대략 $V_{DD}/2$ 까지 키워야 하고, 그것이 $t = \tau \cdot \ln(V_{DD}/2v_0)$ 걸린다.	시간은 표본이 가운데에 얼마나 가까이 떨어졌는지의 로그 다 — 그리고 v_0 는 얼마든지 작을 수 있으므로 t 에 상한이 없다. 대비할 최악의 경우가 없고 분포만 있다.
6	결판나는 데 t_r 보다 오래 걸리는 창 안에 표본이 떨어질 확률은 $e^{-t_r/\tau}$ 로 준다.	클럭 주기 안에서 도착이 고르고 v_0 가 에지에 얼마나 가까웠는지에 비례하므로, 지수가 곧바로 나온다.



입력이 0 -> 위(PMOS)가 켜짐 -> Y 를 VDD 로 당겨 올린다
 입력이 1 -> 아래(NMOS)가 켜짐 -> Y 를 GND 로 당겨 내린다
 둘이 동시에 켜지지 않는다 -> 정상상태 전류가 0 이다

그림 30. 고리를 이룬 두 인버터 가운데 하나. 준안정점에서 그것은 자기 스위칭 문턱에 앉아 있고 두 소자가 모두 포화에 있다 — 그 인버터가 가질 수 있는 가장 이득이 크고, 가장 전류가 크고, 가장 불안정한 동작점이다.

저 유도에서 결론 둘이 떨어지고 둘 다 실무에서 중요하다. 첫째, τ 는 소신호 양이다. 그래서 이득이 나빠지는 바로 그 자리에서 나빠진다 — 느린 코너, 낮은 전압, 높은 온도. 둘째, 준안정점은 플롭이 전류를 가장 많이 태우는 자리다 — 두 소자가 다 켜져 있다. 자주 준안정에 빠지는 설계는 논리적으로만 실패하는 것이 아니라, 예산에 없던 전류까지 끌어간다.

T3.2 MTBF, 그리고 그 안의 기호 하나하나가 실제로 무엇인가

MTBF (mean time between failures) — 동기화기가 주어진 시간 안에 결판내지 못하는 사건 둘 사이의 평균 시간. **보장이 아니라 비율이다** — MTBF 1000 년은 첫 실패가 1000 년째에 난다는 뜻이 아니라, 실패가 그 평균을 갖는 포아송이라는 뜻이다.

MTBF 식

$$MTBF = e^{t_r/\tau} / (T_0 \cdot f_c \cdot f_d)$$

t_r — 해결 시간: 무엇인가가 읽기 전에 동기화기 플롭에게 정착하라고 준 시간. 한 단 건넘이라면 T_{clk} 에서 그 플롭 자신의 t_{cq} 를 빼고, 그것을 읽는 쪽의 셋업을 빼고, 사이의 논리를 뺀 것이다. τ — 결정 시간상수, T3.1 에서 왔고 여기서는 25 ps. T_0 — 표본이 불안정에 빠질 수 있는 구경의 실효 폭, 여기서는 1 ps. 둘 다 유도된 것이 아니라 셀마다 코너마다 켜 것이다. f_c — 표본화(목적지) 클럭 주파수. f_d — 비동기 입력이 변하는 비율.

지수에 들어 있는 것은 t_r 뿐이고, 이 식의 공학적 내용은 전부 그것이다: **당신이 할 수 있는 것은 전부 선형이고, 당신이 제어할 수 없는 것이 지수다.** f_d 를 두 배로 하면 MTBF 가 반이 된다. 동기화기 단을 하나 더하면 지수가 곱해진다.

어디에 쓰나	설계의 모든 비동기 경계: 바깥 인터럽트, 리셋 해제, 누름 버튼, 클럭 도메인을 건너가는 FIFO 포인터, PHY 의 복원 클럭 도메인이 코어와 만나는 자리.
언제 쓰나	아키텍처 단계에서 깊이를 고르고, 사인오프에서 수를 적는다. MTBF 와 그것을 계산한 조건 없이 'CDC 안전' 이라고 적은 IP 데이터시트는 아무것도 주장하지 않은 것이다.
어떻게 쓰나	τ 와 T_0 를 라이브러리의 동기화기 셀 특성화에서 받는다(코너마다 공개돼 있다). t_r 은 이상적인 클럭 주기가 아니라 실제 경로에서 낸다. 그리고 MTBF 를 느린 코너에서 적는다. τ 가 거기서 가장 나쁘기 때문이다.

실제 코드는 이렇게 생겼다

```
// The synchroniser, written so the tools recognise it.
// The attribute is not decoration: it tells the tool to keep the two flops
// adjacent and out of retiming, and it tells CDC checkers this is intentional.
(* ASYNC_REG = "TRUE" *) reg [1:0] sync_q;          // Xilinx; Intel: altera_attribute
always_ff @(posedge clk_dst or negedge rst_n) begin
    if (!rst_n) sync_q <= 2'b00;
    else      sync_q <= {sync_q[0], async_in};    // 2-stage
end
wire safe_in = sync_q[1];

// And the constraint, without which STA tries to time an impossible path:
// set_max_delay -datapath_only 2.0 -from [get_pins u_src/*/Q] -to [get_pins u_sync/sync_q_reg[0]/D]
// NOT set_false_path -- a false path lets the tool put unbounded wire delay there,
// and then the first stage's own setup window is no longer what you assumed.
```

여기서 틀린다. 건넘에 `set_false_path` 를 거는 것이 가장 흔한 CDC 제약 잘못이다. 타이밍 보고서를 조용하게 만들지만, 동기화기로 들어가는 배선 지연의 한계도 같이 없앤다 — 그래서 원천이 목적지 예지에 얼마든지 가까이 변할 수 있게 되고 게다가 버스의 두 비트 사이 스큐에 상한이 없어진다. `set_max_delay -datapath_only` 를 쓰라.

표 76. 클럭 주파수와 동기화기 깊이에 따른 MTBF ($\tau = 25$ ps, $T_0 = 1$ ps, $f_d = f_c/10$, 부대비용 15 ps)

f_clk	단마다 t _r	1 단	2 단	3 단
0.5 GHz	1,85 ps	1.7e+18 yr	2.4e+50 yr	3.3e+82 yr
1 GHz	85 ps	1.85 년	1.1e+15 yr	6.3e+29 yr
2 GHz	35 ps	9.53×10 ⁻¹⁰ 년	0 년	1,378 년

예제 12. 왜 2 단이 기본이고, 어디서부터 그것으로 모자라나

주어진 것. 목적지 클럭 1 GHz, 100 MHz 로 변하는 입력, 단마다 부대비용 15 ps 이므로 t_r = 85 ps.

푸는 법. 깊이마다 수를 식에 한 번씩 넣는다. 단마다 자기 몫의 t_r 을 통째로 받으므로 지수가 단 수에 비례해 커진다.

답. 1 단: 1.85 년 — 건넌 하나당 몇 해에 한 번 실패한다. 2 단: 1.1e+15 yr. 3 단: 6.3e+29 yr.

표 77. 1 GHz 에서 같은 2 단 건넌, τ 가 나빠질 때

τ	MTBF
1×	1.08e+15 yr
1.2×	1.34e+09 yr
1.5×	1.54e+05 yr
2×	1.85 yr

여기서 틀린다. 2 단 수치를 코너의 성질이 아니라 설계의 성질로 읽는 것. τ 가 겨우 2 배 나빠진 값으로 — 느린 코너, 낮은 전압에서 충분히 있을 수 있는 열화다 — 다시 계산하면 같은 2 단 건넌이 **1.85 yr** 을 준다. 지수가 무너진 것이다. **MTBF 는 당신이 제어하지 못하는 바로 그 하나의 매개변수에 지수적으로 민감하다.** 그 수를 느린 코너에서 적어야 하는 까닭이고, 세 번째 단이 있는 까닭이다.

2 GHz 에서 그 표가 멈춰 볼 만한 것을 말한다. **1 단** 건넌의 MTBF 가 0.0301 초다. 해가 아니라 초다. 그 주파수에서 1 단으로 100 년에 닿으려면 t_r = 984.4 ps 가 있어야 하는데, 그것은 클럭 주기 자체(500 ps)보다 길다. **한 주기로는 요구를 아예 못 맞춘다** — 깊이가 안전 여유이기를 그만두고 회로가 도는 유일한 방법이 되는 영역이다.

T3.3 동기화기는 CDC 의 해답이 아니다

지금까지는 전부 한 비트 이야기다. 실제로 실리콘까지 닿는 실패는 MTBF 를 잘못 잡아서 나는 일이 거의 없다. 버스의 비트마다 제대로 동기화해 놓고도 진 설계자들에게서 난다.

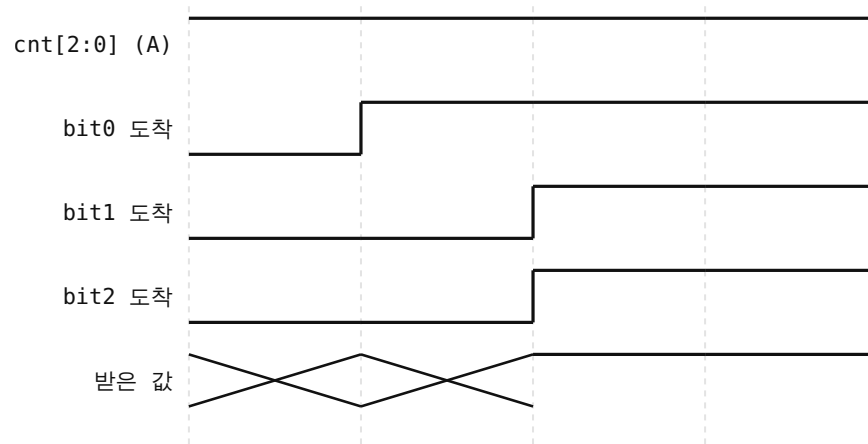


그림 31. 각자의 동기화기를 거쳐 건너는 두 비트. 비트 하나 하나는 안전하다. 그 쌍은 안전하지 않다. 하나는 이번 주기에 결판나고 다른 하나는 다음 주기에 결판나서, 받는 쪽은 한 번도 보내진 적 없는 값을 본다.

다중 비트 문제, 그리고 빠져나가는 세 길

서로 독립인 동기화기는 서로 독립으로 결정한다. 같은 원천 주기에 두 비트가 변하면 각각이 경계의 어느 쪽으로든 결판날 수 있으므로, 목적지는 중간값을 볼 수 있다 — **원천이 한 번도 몰지 않은 코드**. 이진 계수기가 0111 → 1000 으로 건너가는 것을 1111 로도 0000 으로도, 그 사이 아무것으로도 볼 수 있다. 어떤 MTBF 계산도 이것을 못 잡는다. 플롭은 하나하나 다 옳게 동작했다.

어디에 쓰나	클럭 경계를 건너는 모든 버스, 계수기, 포인터, 상태 워드, 설정 필드. 실제로는 FIFO 포인터, 인터럽트 상태 레지스터, 그리고 느린 설정 도메인이 쓰고 빠른 데이터패스가 읽는 '모드' 필드.
언제 쓰나	비트가 하나를 넘는 순간부터. 3 비트라고 32 비트보다 안전하지 않다 — 중요한 것은 폭이 아니라 동시성이다.
어떻게 쓰나	하나를 고르라. (1) 그레이 코드 — 하나 셀 때마다 한 비트만 변하므로 중간에 잡힌 표본은 옛 값 아니면 새 값이고 둘 다 올바르다. 계수기와 포인터에는 되고 임의의 데이터에는 안 된다. (2) 핸드셰이크 — 데이터를 안정하게 붙들고 요청 비트 하나만 건넨다. 받는 쪽이 응답한다. 왕복 지연이 들지만 무엇에든 된다. (3) 비동기 FIFO — 그레이 코드 포인터를 갖는 이중 포트 메모리. (1) 과 (2) 를 합친 것이고, 흐르는 데이터에는 이것을 내보낸다.

실제 코드는 이렇게 생겼다

```
// Gray-coded pointer crossing -- the standard async FIFO core.
// The pointer is converted to gray BEFORE crossing and back AFTER.
function automatic [PW:0] bin2gray(input [PW:0] b); return b ^ (b >> 1); endfunction

always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wbin, wgray} <= '0;
    else        {wbin, wgray} <= {wbin_next, bin2gray(wbin_next)};

// crossing: gray value only, two stages, one synchroniser per bit is now SAFE
// because at most one bit differs between consecutive gray values.
(* ASYNC_REG = "TRUE" *) reg [PW:0] wq1_rgray, wq2_rgray;
always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wq2_rgray, wq1_rgray} <= '0;
    else        {wq2_rgray, wq1_rgray} <= {wq1_rgray, rgray};

// FULL is computed from the SYNCHRONISED read pointer, so it is CONSERVATIVE:
// it can be asserted when the FIFO has already been drained, never the reverse.
assign wfull = (wgray_next == {~wq2_rgray[PW:PW-1], wq2_rgray[PW-2:0]});
```

여기서 틀린다. 그레이 코드는 포인터를 지키지 데이터를 지키지 않는다. 비동기 FIFO 의 데이터 경로는 이중 포트 RAM 이고, 그것이 안전한 까닭은 오직 읽기 주소가 쓰기 쪽이 아직 다 쓰지 못한 자리에 닿는 것이 허용되지 않기 때문이다 — 보수적인 full/empty 비교가 사 주는 것이 그것이다. 그레이 코드 수법을 데이터 버스에 베풀 놓아도 아무 소용이 없다. 이어지는 데이터 워드는 여러 비트가 한꺼번에 다르다.

실제로 난 일

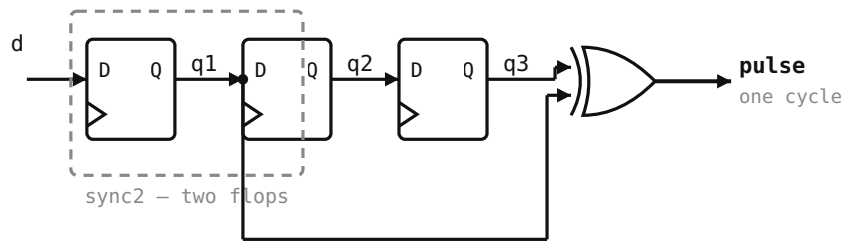
건넌 건 준안정 사건이 **하나도** 없이도 실패할 수 있다. 원천 펄스가 목적지 클럭 주기 하나보다 짧으면 목적지는 그것을 영영 표본화하지 못할 수 있다 — 동기화기는 매번 깨끗하게 옛 값으로 결판난다. 이것은 확률적 실패가 아니라 확

정적 실패이고, MTBF 분석에는 안 보인다. 그것을 막는 규칙: **느린 도메인으로 건너가는 신호는 목적지 주기의 1.5 배 이상 붙들어야 한다.** 아니면 레벨 토글로 바꾸고 돌아오는 핸드셰이크를 둔다.

표 78. 건넌 방식마다 값과 지키는 것

방식	지연 (목적지 주기)	처리량	지키는 것	실패하는 때
2 플롭 동기화기	2	주기당 1 비트	레벨이 안정한 한 비트	한 비트를 넘게 같이 변할 때. 1.5 주기보다 짧은 펄스
펄스 동기화기 (토글)	2-3	3 주기당 사건 1	어떤 비율에서든 단일 사건	사건이 왕복보다 빨리 올 때
그레이 포인터	2	주기당 1 증가	단조 계수기와 포인터	임의의 데이터에 쓸 때
req/ack 핸드셰이크	왕복 4-6	왕복당 1 워드	폭에 관계없이 임의의 데이터	지연 예산이 왕복을 못 삼킬 때
비동기 FIFO	2-3 + 채움	주기당 1 워드 지속	흐르는 데이터, 속도 불일치	깊이가 그 버스트 모양에 비해 작을 때
준정적 (설정 필드)	0	한 번 쓴다	쓰는 쪽이 놓고 있거나 리셋 중일 때만 변하는 필드	데이터패스가 도는 동안 누가 그것을 쓸 때

준정적신호 (quasi-static signal) — 목적지가 그것을 쓰지 않는 동안에만 변하도록 **규약으로 보장된** 건넌 신호 — 보통 블록이 켜지기 전에 쓰는 설정 필드다. 동기화기는 필요 없지만, **데이터시트에 적힌 규칙**과 그렇다고 말하는 CDC 예외는 필요하다. 적어 두지 않은 준정적 가정이야말로 IP 가 실험실에서는 되고 고객 시스템에서는 안 되는 길이다.



Different input-output levels across one flop → exactly one cycle of dif

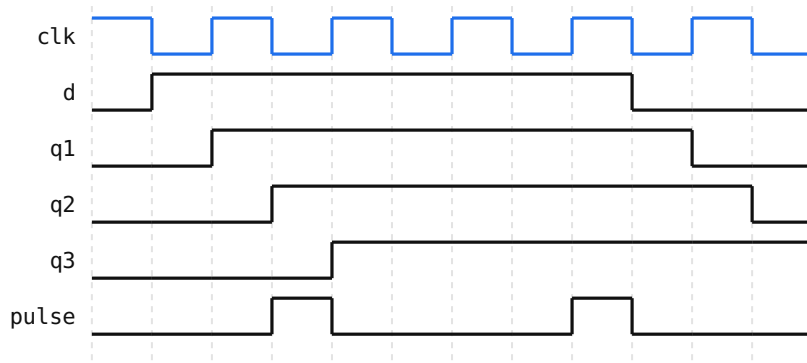
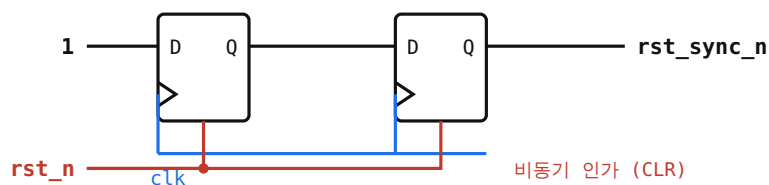


그림 32. 한 주기짜리 사건을 클럭 경계 너머로 나르는 표준적인 방법. 플롭 둘이 준안정을 걸어 내고, 세 번째가 앞의 값을 붙든다. 그 둘의 XOR 가 어떤 레벨 변환든 목적지 도메인의 정확히 한 펄스로 바꾼다. 파형이 그 증거다. 넓은 펄스가 들어가고 한 주기짜리 펄스가 나오는데, 양쪽 에지에서 다 그렇다.

T3.4 리셋: 모두가 잊는 건넌

리셋은 본디 비동기다 — 어떤 클럭도 돌기 전에 동작해야 한다 — 그리고 어쩔 수 없이 동기이기도 하다. 엉뚱한 순간에 풀면 플롭의 리커버리 시간을 어겨서 상태 기계를 첫 주기에 준안정 상태로 만들기 때문이다. 해법은 모든 설계에서 같고, 한 문장으로 외워 둘 값어치가 있다: **비동기로 걸고, 동기로 푼다.**



인가는 즉시 (클럭 없이도 리셋이 걸린다)
해제는 클럭에 맞춰 (두 클럭 뒤) -- recovery/removal 위반을 막는다

그림 33. 리셋 동기화기. 비동기로 거는 경로에는 클럭이 하나도 없다. 푸는 경로는 목적지 도메인으로 클럭되는 플롭 둘을 지난다.

리커버리 · 리무벌 (recovery and removal)

리커버리 시간은 리셋 **해제**에 걸리는 셋업 제약이다. 리셋은 활성 클럭 에지보다 최소한 t_{recovery} 앞서 풀려야 한다. 리무벌 시간은 그에 대응하는 홀드 제약이다. 다른 타이밍 이음과 똑같이 특성화되어 .lib 에 들어 있다 — 그 말은 STA 가 그것을 **그 플롭에서 리셋이 타이밍이 걸린 동기 신호일 때만** 검사한다는 뜻이다.

동기화기 없이 풀리는 비동기 리셋은 클럭과 그런 관계가 없으므로 STA 는 아무것도 보고하지 않고, 실패는 전원을 올린 뒤 상태 기계가 가끔 불법 상태에서 출발하는 꼴로 나타난다 — 대개 한 코너에서, 어떤 보드에서만, 어떤 전원 상승 모양에서만.

어디에 쓰나	모든 클럭 도메인. 클럭이 셋인 설계는 리셋 동기화기가 셋 필요하고, 각각 자기 도메인으로 클럭되며 모두 같은 비동기 원천을 받는다.
언제 쓰나	늘. 값은 도메인마다 플롭 돌리고, 그것이 막아 주는 실패는 재현하기 가장 어려운 부류다.
어떻게 쓰나	거는 것은 바로 통과시키고(클럭 없이), 푸는 것은 플롭 둘을 거치게 한다. 그러고 나서 제약한다. 해제는 이제 동기 신호이므로 STA 가 그것이 닿는 모든 플롭에서 리커버리/리무벌을 검사한다.

실제 코드는 이렇게 생겼다

```
// Reset synchroniser -- one per clock domain.
module reset_sync (input clk, input arst_n, output rst_n);
    (* ASYNC_REG = "TRUE" *) reg [1:0] q;
    always_ff @(posedge clk or negedge arst_n)
        if (!arst_n) q <= 2'b00;          // ASSERT: asynchronous, no clock needed
        else        q <= {q[0], 1'b1};    // RELEASE: two clocked stages
    assign rst_n = q[1];
endmodule

// SDC: the asynchronous assertion is not a timed path; the release is.
set_false_path -from [get_ports arst_n] -to [get_pins u_rsync/q_reg[0]/CLR]
// and every flop fed by rst_n gets a normal recovery/removal check -- verify with:
// report_timing -delay_type max -check_type recovery -from [get_pins u_rsync/q_reg[1]/Q]
```

여기서 틀린다. 동기화기 자신의 플롭을 동기화된 리셋으로 리셋하는 것은 빠져나갈 길 없는 고리다. 해제 에지를 놓치면 블록이 영영 리셋에서 못 나온다. 동기화기는 클리어 핀에 **날것의 비동기** 리셋을 받아야 한다. 리셋 동기화기의 두 플롭이 0 으로 리셋되고 1 을 밀어 넣는 것이지 그 반대가 아닌 까닭도 이것이다.

T3.5 STA 가 못 보는 것, 그리고 CDC 도구가 대신 하는 것

표 79. 도구 둘, 물음 둘

물음	STA	CDC 분석
주어진 클럭 관계에서 이 경로가 셋업을 맞추나?	본다	안 본다
클럭 관계라는 것이 있기는 한가?	있다고 가정한다	이것이 그 일이다
모든 건넌에 동기화기가 있나?	안 본다	본다 (구조적으로)
여러 비트가 같이 건너나?	안 본다	본다 — 재수렴과 버스 일관성 검사
원천이 충분히 오래 안정한가?	안 본다	본다 — 규약 검사, 보통 시뮬레이션의 생성된 어서션으로
이 건넌은 일부러 동기화를 안 한 것인가?	안 본다	당신이 이름과 까닭을 적어 예외로 두었을 때만

예외 파일이야말로 CDC 사인오프에서 그 분석에 뜻이 있었는지를 정하는 부분이다. 예외 이백 개를 달고 돌려서 나온 초록 요약은 증거가 아니다. 그것은 기술자 한 사람의 주장 이백 개다. **IP 납품물은 CDC 예외를 읽을 수 있는 문서로 같이 내보내야 한다.** 하나마다 까닭 한 줄씩. 통합하는 쪽이 그 전부를 물려받는데, 넷리스트에서 그것을 되살려 낼 수는 없기 때문이다.

구조뿐 아니라 시뮬레이션으로도 건넌을 증명하기

구조적 CDC 분석은 동기화기가 있다는 것을 증명한다. 원천이 그 동기화기가 가정하는 규약을 지키는지는 증명하지 못한다 — 요청이 높은 동안 데이터가 안정한지, 펄스가 충분히 긴지, 준정적 필드가 데이터패스가 도는 동안 쓰이지 않는지. 그것들은 **시간적 성질**이고, 손으로 쓰거나 CDC 도구가 생성한 어서션으로 검사한다.

쓸 수 있는 가장 강한 검사는 준안정 주입까지 더한다. 시뮬레이터가 동기화기 출력을 무작위로 한 주기 늦춰서, 건넌이 즉시 결판날 때만 도는 설계가 실리콘이 아니라 회귀에서 실패하게 만든다.

어디에 쓰나

클럭이 하나를 넘는 모든 것의 블록 수준 검증, 그리고 남의 클럭 구조 안으로 팔 생각인 모든 IP.

언제 쓰나

RTL 을 얼릴 때마다 구조적 CDC 를 돌리고, 야간 회귀에서 주입을 돌린다. 둘은 서로 다른 버그를 찾고 어느 쪽도 다른 쪽을 대신하지 못한다.

어떻게 쓰나

어서션을 건넌 모듈에 묶고, 주입을 켜 채 회귀를 돌리고, 결과를 납품물의 검증 절차 건넌 수와 예외 수와 함께 적는다.

실제 코드는 이렇게 생겼다

```
// The protocol the handshake crossing assumes, stated as a checkable property.
property data_stable_while_req;
    @(posedge clk_src) disable iff (!rst_n)
        req |-> ##0 $stable(data) throughout (req [*1:$] ##1 ack_sync);
endproperty
assert property (data_stable_while_req)
    else $error("CDC: data changed while req was asserted -- the receiver may latch a mix");

// Metastability injection (simulation only): the synchroniser may take one extra cycle.
`ifdef CDC_INJECT
    always_ff @(posedge clk_dst)
        if ($urandom_range(0,99) < 5) sync_q[1] <= sync_q[1]; // hold: resolve later
`endif
```

여기서 틀린다. 원천 도메인 신호에 대해 목적지 도메인에서 쓴 어서션은 그 자체가 클럭 도메인 건넌이고, 테스트벤치가 만들어 낸 가짜 실패를 보고할 수 있다. 원천 신호는 원천 도메인에서 표본화하고 (@(posedge clk_src)) 불리언 결과만 건네라.

T3.6 월요일에 무엇을 할 것인가

표 80. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
MTBF · 결정시간상수	모든 비동기 경계	동기화기 깊이, 그리고 데이터시트에 적는 수
동기화기깊이	아키텍처, 건넘마다	기본은 둘. 1 GHz 위이거나 τ 특성화가 시원치 않으면 셋
다중비트문제 · 그레이코드	FIFO 포인터, 계수기, 상태 워드	그 건넘에 부호화가 필요한가, 핸드셰이크인가, FIFO 인가
핸드셰이크 · 비동기FIFO	도메인 사이로 흐르는 데이터	지연과 깊이, 따라서 면적
리셋동기화기 · 리커버리	모든 클럭 도메인	모든 코너에서 블록이 합법 상태로 출발하는가
CDC검증 · 준정적신호	사인오프와 납품물	통합하는 쪽이 물려받는 예외 목록

이 장을 끝내는 수는 둘이고, 들고 갈 것이 그 둘이다. 1 GHz 에서 1 단 건넘은 대략 1.8 년에 한 번 실패하고, τ 가 두 배인 같은 건넘은 2 단으로도 대략 1.8 년에 한 번 실패한다. 첫 번째 수가 동기화기가 존재하는 까닭이고, 두 번째 수가 답을 늘 그 코너와 함께 적어야 하는 까닭이다.

다음 장은 이 장이 내내 가정하고 있던 것 — 표본은 맞거나 틀리거나 둘 중 하나라는 것 — 을 분포로 바꾼다. 잡음은 혼성신호 IP 의 모든 아날로그 측정 아래에 바닥을 깔고, 준안정과 달리 전력·면적과 곧바로 맞바꿀 수 있다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

준안정 자체는 물리이고, 2 플롭 동기화기는 이 글을 읽는 누구보다도 오래됐다. **지금도 청구되고 있는 것은 그 둘레의 배치다:** 다중 비트 건넘을 N 개가 아니라 동기화기 하나로 안전하게 만드는 포인터 부호화(그레이 코드와 그 후에 들). 동작 모드에 따라 동기화기 깊이를 바꾸는 지연 대 MTBF 방식. 자격 신호 하나만 동기화하고 데이터는 경계 너머로 안정하게 붙들어서 데이터를 아예 동기화하지 않는 데이터 경로 건넘. 이 하나하나가 플롭에 대한 논의가 아니라 무엇이 건넘야 하는가에 대한 논의다. 이 장을 이렇게 물으며 읽으라: 내 신호 가운데 정말로 건넘야 하는 것은 무엇이고, 나머지는 무엇이 있으면 한쪽에 머물 수 있는가?

T4. 잡음 — 그리고 그것이 왜 모든 것의 크기를 정하나

잡음은 영리함만으로는 개선할 수 없는 단 하나의 규격이다. 6 dB 마다 용량이 4 배이고, 따라서 전력도 4 배다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 온도 · 확률 · 분산 · 표준편차 · 적분 · 로그 · 주파수 · 트랜스컨덕턴스 · 게이트용량 · 출력저항 · RC지연 · 준안정

이 장에서 새로 나오는 것

열잡음 · 산탄잡음 · 플리커잡음 · 잡음스펙트럼밀도 · 잡음 대역폭 · kTC잡음 · 입력환산잡음 · 플리커코너 · 잡음지수 · 정적잡음여유 · 나비곡선 · 읽기방해 · 최소동작전압 · SNR · SNDR · ENOB

회로에서 재는 모든 것에는 잡음이 얹혀 온다. 그 잡음은 서툰 공학에서 오는 것이 아니다 — 전하의 열운동과 전자가 알갱이라는 사실에서 온다. 그래서 가장 다루기 어려운 종류의 규격이다. 없앨 수는 없고 면적과 전력을 써서 평균으로 깎아 내릴 수만 있는데, 그 환율은 물리가 **비트 하나당 4 배**로 고정해 두었다.

이 장은 실제로 만나게 될 잡음원 셋을 세우고, 혼성신호 IP 의 아날로그 블록 절반의 크기를 정하는 수 하나(kT/C)를 유도한 다음, 잡음 장에서는 드문 일을 한다. 디지털 쪽으로 건너간다. 거기서는 같은 낱말이 전혀 다른 것을 뜻하고, 그 결과 — SRAM 을 읽을 수 없게 되는 전압 — 를 주장하지 않고 ngspice 로 여기서 쟀다.

T4.1 세 가지 원천, 그리고 그중 무엇에 손댈 수 있나

표 81. IC 안의 잡음원을 출처로 가른다

원천	밀도	출처	바꿀 수 있나?
열잡음 (존슨)	$S_V = 4kTR [V^2 / \text{Hz}]$	저항성 요소 안 운반자의 열운동	R 이나 T, 또는 내다보는 대역폭을 바꿔서만
채널 열잡음	$S_i = 4kT\gamma g_m [A^2 / \text{Hz}]$	MOSFET 채널 안의 같은 기구. $\gamma \approx 2/3$ (장채널), 1-2 (단채널)	g_m 을 올리거나(전류가 든다) 거른다
플리커 (1/f)	$S_V = K_f / (C_{ox}WL \cdot f)$	산화막 계면에서의 포획과 방출	있다 — 소자를 크게, PMOS 입력, 초핑, 자동영점
산탄	$S_i = 2qI [A^2 / \text{Hz}]$	장벽을 건너는 전하가 알갱이라는 것(채널이 아니라 접합)	전류를 바꿔서만

유도 — 답이 왜 늘 kT/C 로 끝나나

	이렇게 된다	왜냐하면
1	저항의 열잡음 밀도는 $4kTR$ 이고 주파수와 무관하다.	평형 열역학에서 나온 나이퀴스트의 결과다 — R 말고는 소자 매개변수가 하나도 안 들어 있고, 그래서 이것은 설계 선택이 아니라 바닥이다.
2	그 저항을 축전기 C 와 직렬로 놓는다 — 모든 표본화 스위치가 바로 그것이다.	스위치의 온저항($T1$: 선형 영역)과 표본화 축전기가 직렬이다. 세 번째 요소는 없다.
3	RC 저역통과가 그것을 모양 지우고, 극 하나의 잡음 대역폭은 $\pi/2 \times f_{-3dB} = 1/(4RC)$ 이다.	$1/(1+(f/f_0)^2)$ 를 모든 주파수에 걸쳐 적분하면 f_0 가 아니라 $\pi/2 \cdot f_0$ 가 나온다 — 사람들이 빠뜨리고는 왜 측정이 예측보다 1.6 배 나쁜지 의아해하는 그 인자다.
4	$v_n^2 = 4kTR \times 1/(4RC) = kT/C$.	R 이 사라진다. 표본화된 축전기의 잡음은 스위치와 전혀 무관하다 — C 와 온도에만 달려 있다.
5	3 K 에서 $C = 1$ pF 이면 $v_n = 64.4 \mu V$ rms 다.	외워 둘 값어치가 있는 수 하나. 나머지는 전부 여기서 $1/\sqrt{C}$ 로 비례한다.

kT/C — 축전기의 크기를, 따라서 전력을 정하는 규칙

표본화된 마디는 어떻게 표본화했든 kT/C 의 잡음 전력을 진다. 그것을 N 비트 변환기의 양자화 잡음($LSB/\sqrt{12}$) 아래로 두려면 축전기가 $C \geq 12kT \cdot 2^{2N}/V_{FS}^2$ 를 만족해야 한다. **비트 하나마다 C 가 네 배가 되고**, 정해진 시간 안에 정착하려면 $g_m \propto C$ 이므로 전류도 네 배가 된다.

이 관계 하나가, 16 비트 변환기가 12 비트보다 "조금 더 어려운" 것이 아닌 까닭이다. 용량이 256 배다.

어디에 쓰나	모든 스위치드 커패시터 회로: SAR 과 파이프라인 ADC 의 표본화 회로망, 샘플앤홀드, 스위치드 커패시터 필터와 이득단, 그리고 그것들을 몰아야 하는 기준 버퍼.
언제 쓰나	아날로그 블록 설계의 맨 처음, 트랜지스터 치수를 정하기 전에. 축전기가 먼저다. 그것이 전력 예산을 고정하기 때문이다.
어떻게 쓰나	V_{FS} 와 N 을 정해 C 를 내고, 표본화 구간 안에 정착하도록 드라이버 치수를 정하고($T1$ 의 스위치 R_{on} 을 쓴 $T2$ 의 RC 셀), 기준이 주기마다 그 전하를 댈 수 있는지 확인한다.

실제 코드는 이렇게 생겼다

```
* The measurement that separates kT/C from everything else you did wrong.
* Transient noise in ngspice: sample the same node 1000 times and take the std dev.
.tran 1n 10u
.noise v(out) VIN dec 20 1 1G      $ small-signal, for the spectrum
.measure noise inoise_total_rms param='sqrt(inoise_total)'
* And the closed form to check it against -- ALWAYS have the hand calculation:
*   v_n = sqrt(kT/C) = sqrt(1.38e-23*300/1e-12) = 64.4 uV for C = 1 pF
```

여기서 들린다. kT/C 는 **표본화된** 값에 적용된다. 트래킹 구간에서는 잡음 대역폭을 표본화가 아니라 회로가 정한다 — 그래서 트래킹 중에 켜 트래킹앤홀드는 같은 회로를 홀드 뒤에 켜 것과 다른(보통 더 큰) 수를 보인다. 규격이 뜻하는 쪽이 아닌 쪽을 보고하는 것은 혼하고 비싼 잘못이다.

표 82. 비트당 축전기와 전력 값 ($V_{FS} = 1\text{ V}$, 300 K , 잡음 = 양자화 잡음)

비트 N	LSB	양자화 rms	필요한 C	8 비트 대비
8	$3,906\text{ }\mu\text{V}$	$1,128\text{ }\mu\text{V}$	3.26 fF	$1\times$
10	$977\text{ }\mu\text{V}$	$282\text{ }\mu\text{V}$	52.1 fF	$16\times$
12	$244\text{ }\mu\text{V}$	$70.5\text{ }\mu\text{V}$	834 fF	$256\times$
14	$61\text{ }\mu\text{V}$	$17.6\text{ }\mu\text{V}$	13.3 pF	$4,096\times$
16	$15.3\text{ }\mu\text{V}$	$4.4\text{ }\mu\text{V}$	213 pF	$65,536\times$

T4.2 플리커 잡음, 그리고 아키텍처를 정하는 코너

플리커코너 (flicker corner, f_c) — $1/f$ 밀도가 열잡음 밀도와 같아지는 주파수. 그 아래는 플리커가 지배하고 그 위는 열잡음이 지배한다. 요즘 CMOS 증폭기에서는 NMOS 입력이면 $\sim 100\text{ kHz}$ 에서 몇 MHz 사이 어디든 있고, PMOS 면 보통 한 자릿수 낮다.

신호가 그 코너의 어느 쪽에 있나

플리커 잡음 밀도는 $1/f$ 로 오르므로, f_1 에서 f_2 까지의 **적분된** 전력은 $\ln(f_2/f_1)$ 에 비례한다 — 폭이 아니라 비에 달려 있다. DC 결합 측정은 $f_1 \rightarrow 0$ 이고 적분이 로그로 발산한다: **오래 평균할수록 나빠진다**. 열잡음과 정반대다.

그 사실 하나가 아키텍처 결정을 이끈다. 신호가 DC 근처에 있으면 (온도 센서, 밴드갭 기준, DC 오프셋) 평균은 도움이 안 되고 신호를 옮겨야 한다 — 코너 위 주파수로 잘게 썰어 올려 거기서 증폭하고 다시 내린다. 신호가 이미 코너 위에 있으면(RF 나 고속 링크 전단) 플리커는 상관없고 거기에 아무것도 안 쓴다.

어디에 쓰나

센서 전단, 기준과 바이어스 회로, 비교기 오프셋, DC 정확도가 필요한 모든 측정. 그리고 발진기의 위상 잡음 — 소자 플리커가 가까운 쪽 위상 잡음으로 옮겨 실려 간다.

언제 쓰나

관심 대역에 ~ 1 MHz 아래가 들어가면 언제든. 아키텍처 단계에서 그 물음을 명시적으로 던지라. 다 만든 증폭기에 초핑을 나중에 끼워 넣는 것은 재설계다.

어떻게 쓰나

면적으로 줄이거나($S \propto 1/WL$ — 6 dB 에 면적 4 배), PMOS 입력을 고르거나, 변조로 줄인다: **초핑**(들어갈 때와 나올 때 구형파로 변조한다. 초핑 주파수에 잔여 리플이 남는다) 또는 **상관 이중 표본화**(오프셋을 표본화해 빼낸다. 잡음 접힘과 대역폭을 치른다).

실제 코드는 이렇게 생겼다

```
* Flicker in the model card is one parameter -- and simulators differ.
.model NM NMOS (LEVEL=54 ... KF=1e-25 AF=1.0 EF=1.0)
* KF alone means nothing: the density is KF/(Cox*W*L*f^EF), so a KF taken from
* another PDK, or from a model with a different EF convention, is a WRONG number
* that still simulates. Always sanity-check the corner against measured silicon:
.noise v(out) VIN dec 10 1 100MEG
* then read the point where the 1/f asymptote crosses the flat part.
```

여기서 틀린다. 초핑은 잡음을 줄이지 않는다. **신호를 잡음에서 멀리 옮기는 것**이다. 열잡음 바닥은 그대로이고, 초퍼는 초핑 주파수와 그 고조파에 자기 나름의 스위칭 부산물을 더한다. 저주파 잡음 수치는 아름답게 나오고 f_{chop} 에 스퍼가 있는 설계는, 그 스퍼가 대역 안에 떨어지면 아무것도 개선한 것이 없다.

T4.3 또 다른 잡음: 정적 잡음 여유를 잔다

디지털 쪽에서 '잡음'이라는 낱말은 다른 것을 뜻하고, 그 양은 스펙트럼 밀도가 아니라 **전압**이다. 저장 마디가 뒤집히기 전에 얼마만큼의 DC 교란을 삼킬 수 있는가. 구조는 T3 에서 준안정을 낳았던 그 교차 결합 쌍 그대로이고, 동적으로가 아니라 정적으로 볼 뿐이다.

정적잡음여유 (SNM, static noise margin) — 교차 결합 쌍의 두 저장 마디에 직렬로 끼워 넣어도 쌍안정을 무너뜨리지 않는 가장 큰 DC 전압. 기하로는 두 인버터 전달 곡선이 이루는 나비 곡선 안에 들어가는 가장 큰 정사각형의 한 변이다.

실제로 난 일

아래에서 쓴 추출기는 맞기 전에 **두 번 틀렸고**, 두 번 다 수가 그럴듯해 보였다. 첫 판은 내접 정사각형이 아니라 가로간격을 재서 $\text{hold SNM} \approx 0.5 \cdot V_{\text{DD}}$ 를 냈다 — 너무 높다. 둘째 판은 액세스 트랜지스터의 게이트가 워드선이 아니라 비트선에 물려 있었다. 그랬더니 셀비가 오를수록 read SNM 이 떨어졌다. 거꾸로다. **버그를 잡은 것은 값이 아니**

라 경향이였다. 이 저장소가 굴러가는 규칙이 그것이다: 측정을 믿기 전에, 내가 이해하는 것을 바꿨을 때 그것이 어느 쪽으로 움직이는지 보라.

그리고 나서 추출기를 두 극한으로 자해검사했다. 이득이 아주 큰 인버터는 $SNM \rightarrow V_{DD}/2$ 를 내야 하고(0.4825 를 냈다), 이득이 1 인 '인버터' 는 0 을 내야 한다(0.0 을 냈다). 그 둘을 다 지나고 나서야 아래 셀 수치를 적었다.

표 83. 6T 셀의 측정된 SNM (ngspice, LEVEL=1 모형, $V_{DD} = 1.0\text{ V}$, $V_{th} = 0.35\text{ V}$) — β 는 셀비 $W_{pulldown} / W_{access}$

β	Hold SNM	Read SNM	Read / Hold
1	0.421 V	0.194 V	0.46
1.5	0.417 V	0.207 V	0.5
2	0.414 V	0.226 V	0.55
2.5	0.411 V	0.241 V	0.59
3	0.409 V	0.253 V	0.62

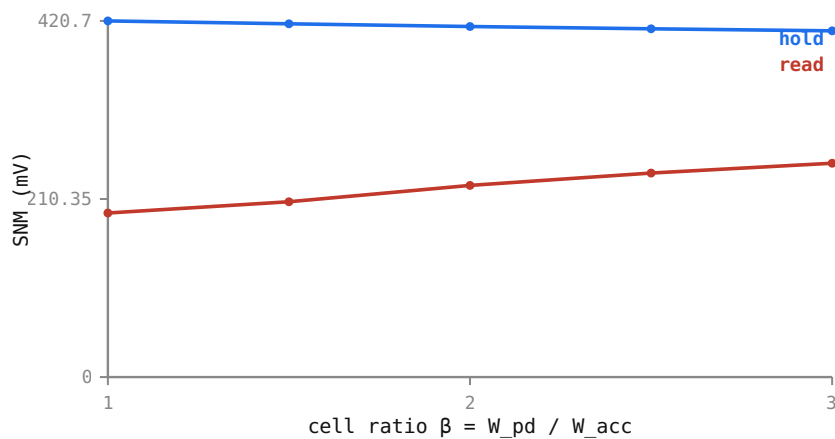


그림 34. 읽기는 셀 여유의 절반 넘게를 먹고, 그것을 되사는 구조적인 길은 액세스 소자에 견준 풀다운을 강하게 하는 것뿐이다 — 6T 셀이 최소 소자로 만들어지지 않는 까닭이다.

유도 — 셀을 읽는 것이 왜 그 셀을 거의 부수는 일인가

	이렇게 된다	왜냐하면
1	읽으려면 두 비트선이 높게 미리 충전된 상태에서 워드선이 액세스 트랜지스터 둘을 켜다.	읽기 포트를 따로 두지 않고 셀을 감지하는 길이 그것뿐이다.
2	이제 '0' 쪽의 액세스 트랜지스터가 그 마디를 위로 끌어올린다. 셀 자신의 풀다운 NMOS 와 이루는 분압기를 거쳐서.	V_{DD} 와 접지 사이에 NMOS 둘이 직렬이고 저장 마디가 가운데 있다 — 그 마디가 0 V 에 머물 수가 없다.
3	저항 극한에서 '0' 마디가 대략 $V_{DD} \cdot (1/(1+\beta))$ 까지 오른다. 그러니 β 가 2 여도 벌써 3 분의 1 만큼 올라간다.	그 시점에 두 소자가 다 선형 영역에 있고, 둘의 세기 비가 곧 분압비다. 셀비가 설계 매개변수로 존재하는 까닭은 오직 이것이다.
4	그 상승이 여유를 곧바로 먹는다. $\beta = 2$ 에서 측정된 read SNM 은 0.226 V 이고 hold SNM 은 0.414 V 다 — 워드선이 오르는 순간 여유의 45.3 % 가 사라진다.	주장이 아니라 젠 것이다 — 텍과 추출기가 이 장의 소스에 있다.
5	β 를 올리면 read SNM 이 오르고 hold SNM 은 조금 내려간다 .	짚다: 같은 쓸기에서 hold 가 0.421 V 에서 0.409 V 로 떨어진다. 풀다운이 강해지면 인버터마디의 전이점이 $V_{DD}/2$ 에서 밀려나기 때문이다. 공짜인 방향은 없다 . 그래서 이것이 규칙이 아니라 설계점인 것이다.

V_{min} — 논리가 아니라 메모리가 칩을 멈추는 전압

Read SNM 은 전원을 내릴 때 점잖게 나빠지지 않는다. $\beta = 2$ 인 같은 셀에서 재면 1.0 V 에서 0.7 V 까지는 0.22 V 언저리를 지키다가 **무너진다**. 0.6 V 에서 0.145 V, 0.5 V 에서 0.087 V. 그 무릎은 V_{DD} 가 $2V_{th}$ (여기서는 0.7 V)에 가까워지는 자리에 앉는다 — 그 아래에서는 소자가 약반전에 있고 분압비가 지수 전류에 달리게 되며, 불일치(T12)가 아슬아슬한 셀을 실패하는 셀로 바꾼다.

그래서 **SRAM 이 칩의 최소 동작 전압을 정한다**. DVFS 방식이 배열의 V_{min} 위에서 멈추거나 배열에 자기 전원을 따로 주는 까닭이다.

어디에 쓰나	칩 안에 SRAM 이 있고 전압점이 하나를 넘는 모든 SoC: DVFS 범위 전체, 잠 모드의 보존 전압, 그리고 사인오프의 저전압 코너.
언제 쓰나	전력 모드가 제안되는 즉시. '0.6 V 까지 내려도 되나' 는 논리의 물음이기 전에 SRAM 의 물음이다.
어떻게 쓰나	보조 회로로 V_{min} 을 올리거나(읽기에는 워드선 언더드라이브, 쓰기에는 음의 비트선), 배열의 전원을 가르거나, 읽기 경로가 저장 마디에 닿지 않는 8T 셀을 쓴다.

실제 코드는 이렇게 생겼다

```
* The sweep that produces the number above -- run it at every corner,
* not just typical, because Vmin is a MISMATCH-limited quantity (see T12).
.param beta=2.0
.dc VIN 0 VDD 0.004 VDD 1.0 0.5 -0.1      $ nested sweep: input, then supply
.print dc v(vr)
* Extraction is NOT a .measure one-liner: you need the butterfly and the largest
* inscribed square. Script it, and SELF-CHECK the extractor on two limits
* (very high gain -> VDD/2, gain 1 -> 0) before trusting any cell number.
```

여기서 틀린다. 전형 코너에서 전 SNM 은 V_{min} 에 거의 쓸모가 없다. 배열은 수백만 개 가운데 **가장 나쁜** 셀에서 실패하므로, 중요한 양은 꼬리다. V_{min} 은 대략 -5σ 에서 -6σ 셀이 정하고, 그 말은 분석이 공칭 DC 쓸기가 아니라 중요한 표본화를 쓴 몬테카를로라는 뜻이다. 위의 쓸기는 모양을 알려 줄 뿐 수율을 알려 주지 않는다.

T4.4 볼트에서 데이터시트의 수로: SNR, SNDR, ENOB

표 84. 세 수, 그리고 각각이 감추는 것

양	정의	감추는 것
SNR	신호 전력 / 잡음 전력, 왜곡은 뺀다	왜곡 . 선형성이 끔찍한 변환기도 SNR 은 훌륭할 수 있다
SNDR (SINAD)	신호 / (잡음 + 모든 왜곡)	둘 중 무엇이 지배하는지 — 스펙트럼도 늘 같이 적으라
ENOB	$(\text{SNDR}_{\text{dB}} - 1.76) / 6.02$	그것이 전 것이 아니라 유도된 것이라는 사실. ENOB 은 단위만 같아입은 SNDR 이다. 입력 주파수와 진폭 없이 적힌 ENOB 은 측정이 아니다
SFDR	신호 / 가장 큰 스퍼 하나	광대역 잡음을 통째로 — 깨끗해 보이는 SFDR 에 나쁜 잡음 바닥은 흔하다

예제 13. 잡음 64 μV 는 몇 비트를 허락하나?

주어진 것. 전체 눈금 1 V 인 변환기의 표본화 회로망에 $C = 1 \text{ pF}$ 이 있어 표본화된 잡음이 $64.4 \mu\text{V rms}$ 이고, 나머지는 전부 완벽하다고 하자.

푸는 법. 전체 눈금 사인파의 rms 진폭은 $V_{FS}/(2\sqrt{2})$ 다. $\text{SNR} = 20 \cdot \log_{10}(\text{신호 rms} / \text{잡음 rms})$ 이고, $\text{ENOB} = (\text{SNR} - 1.76)/6.02$ 다.

답. $\text{SNR} = 74.8 \text{ dB}$ 이고 $\text{ENOB} = 12.1$ 비트다. **표본화 축전기 1 pF 은 ~12 비트짜리 부품이다.** 나머지 설계가 아무리 좋아도 그렇다.

여기서 틀린다. 표본화 회로망이 보통 두 번 표본화된다는 것(입력과 기준)을 잊는 것, 그리고 축전기를 차동 쌍으로 두면 잡음 전력이 두 배가 되면서 신호 진폭도 두 배가 된다는 것 — 둘이 일부 상쇄하고, 나오는 인자는 위상에 달려 있다. 그 구체적인 회로망을 적어라. 외워 둔 인자를 재활용하지 마라.

T4.5 월요일에 무엇을 할 것인가

표 85. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
kTC잡음	트랜지스터 치수를 정하기 전의 모든 스위치드 커패시터 블록	표본화 축전기, 따라서 전류 예산
열잡음 · 잡음대역폭	증폭기와 기준 설계	입력단에 g_m 이 얼마나 필요한가
플리커코너	DC 근처 신호 경로의 아키텍처	이 설계에 초핑이나 자동영점이 아예 필요한가
정적잡음여유 · 나비 곡선	SRAM, 모든 교차 결합 감지증폭기와 래치	셀비 β , 그리고 읽기가 읽은 것을 부수는가
최소동작전압	DVFS 와 보존 모드	칩을 돌려도 되는 가장 낮은 전압
SNDR · ENOB	데이터시트와 고객과의 대화	고객이 경쟁사와 견줄 단 하나의 수

이 장에서 들고 나갈 쟁 결과 둘. 표본화 축전기 1 pF 은 12.1 비트짜리 부품이다 — 트랜지스터를 하나도 그리기 전에 축전기가 해상도를 정해 버린다. 그리고 6T SRAM 셀을 읽는 것은 셀비 2 에서 그 정적 여유의 45.3 % 를 치른다. 그 셀비가 존재하는 까닭이고, 논리가 아니라 배열이 칩을 돌릴 수 있는 가장 낮은 전압을 정하는 까닭이다.

다음 장은 이 장이 순간적이라고 다룬 표본화 동작에 유한한 구경과 지터와 스펙트럼을 준다 — 고속 변환기의 잡음 바닥이 실제로 어디서 오는지가 거기다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

잡음은 열역학이다. kT 에 특허를 내는 사람은 없다. 청구되는 것은 그 4 배를 되사는 모든 배치다 — 상관 이중 표본화, 초핑, 자동영점, 잡음 성형, 그리고 회로가 잡음 예산을 신호가 있는 자리에만 쓰게 해 주는 일군의 기법들. 디지털 쪽에서는 같은 모양이 SRAM 보조 회로로 나타난다: 음의 비트선, 워드선 언더드라이브, 읽고-고치고-쓰기 보조 — 전부 맨 셀이 여유를 하나도 못 갖는 전압에서 셀이 여유를 지키게 해 주는 배치다. 이 장을 이렇게 물으며 읽으라: 나는 어디서 4 배 규칙을 치르고 있고, 무엇이 있으면 그것을 안 치를 수 있는가?

T5. 표본화 — 연속 신호가 수가 되는 자리

클럭 둘이 모든 것을 정한다. 언제 표본화하는가, 그리고 그 '언제' 를 얼마나 정확히 아는가. 오늘 팔리는 모든 고속 변환기를 묶는 것은 두 번째다.

읽기 전에 알아야 하는 것

전압 · 주파수 · 위상 · 적분 · 미분 · 확률 · 표준편차 · 로그 · RC지연 · 슬루 · kTC잡음 · 열잡음 · SNR · SNDR · ENOB · 잡음대역폭

이 장에서 새로 나오는 것

표본화정리 · 엘리어싱 · 나이퀴스트 · 대역통과표본화 · 구경시간 · 구경지터 · 지터한계SNR · 추적유지 · 트래킹대역폭 · 차지인젝션 · 클럭피드스루 · 부트스트랩스위치 · 오버샘플링비 · 잡음성형 · 데시메이션

표본화는 보통 이상화로 가르친다 — 임펄스 열을 곱하고, 스펙트럼이 되풀이되는 것을 보고, 겹치지 않게 한다. 그 이야기는 맞고, 하드웨어를 묶는 것은 그 이야기가 아니다. 진짜 표본화기는 스위치를 유한한 시간 동안, 유한한 저항을 통해, 몇 피코초 안에서만 알 수 있는 순간에 닫아서, T4 의 kT/C 가 이미 크기를 정해 놓은 축전기에 엮는다. 그 네 낱말 — 유한, 유한, 순간, 축전기 — 하나하나가 해상도를 치르고, 몇백 메가헤르츠 위에서 지배하는 것은 세 번째다.

T5.1 엘리어싱은 지켜야 하는 정리가 아니라 사야 하는 필터다

유도 — 임펄스 열에서 접힌 스펙트럼까지

	이렇게 된다	왜냐하면
1	f_s 로 표본화하는 것은 신호에 $T_s = 1/f_s$ 간격의 임펄스 열을 곱하는 것이다.	이상화다. 엘리어싱에 관한 모든 것이 이 한 걸음에서 따라 나온다. 비이상성은 나중에 오고 이것을 바꾸지 않는다.
2	시간에서의 곱은 주파수에서의 합성곱이고, 임펄스 열의 스펙트럼은 그 자체가 f_s 간격의 임펄스 열이다.	그래서 입력 스펙트럼이 f_s 의 모든 배수 자리로 복사된다.
3	입력이 $f_s/2$ 보다 좁은 대역에 갇혀 있지 않으면 복사본이 겹친다.	나이퀴스트. 그것이 말하지 않는 것을 눈여겨보라. 대역이 어디에 얹는지는 아무 말도 안 하고, 얼마나 넓은지만 말한다.
4	$f_s/2$ 위의 모든 것이 아래로 접히고, 그 뒤로는 진짜 신호와 구별되지 않는다.	어떤 디지털 처리로도 되돌릴 수 없다 — 둘을 가르던 정보가 표본화기에서 이미 없어졌다. 안티엘리어스 필터가 아날로그 부품이고 소프트웨어로 옮길 수 없는 까닭이다.
5	잡음도 접힌다, 같은 규칙으로.	대역폭 5 GHz 인 전단을 200 MHz 로 표본화하면 열잡음 50 대역이 신호 위로 접힌다 — 17 dB 벌점이다. 필요 없는 대역폭은 공짜가 아니다. 값을 치르는 잡음이다.

대역통과 표본화 — 접힘을 일부러 쓰기

나이퀴스트는 대역의 폭을 묶지 그 자리를 묶지 않는다. 2.4 GHz 중심의 100 MHz 를 차지하는 신호는 250 MS/s 로 표본화할 수 있다. 대역이 나이퀴스트 구역 하나 안에 온전히 들어가고 표본화기의 입력 대역폭이 2.4 GHz 까지 닿기만 하면 된다. 그 알리아스가 곧 원하는 신호이고, 공짜로 아래로 옮겨져 있다.

값은 분명하다. 표본화 **구경**이 표본화율은 250 MHz 인데도 2.4 GHz 에 걸맞게 좋아야 한다 — 지터는 클럭 속도가 아니라 입력 주파수에 환산된다. 그 맞바꿈(믹서는 없애지만 훨씬 어려운 클럭)이 공학적 결정의 전부다.

어디에 쓰나	IF 에서 디지털화하는 RF 수신기, 소프트웨어 정의 라디오 전단, 그리고 높은 주파수에 얹은 좁은 대역을 잡아야 하는 모든 계측기.
언제 쓰나	대역이 중심 주파수에 견주어 좁고, 그러지 않으면 하향 변환 믹서와 그 LO 값을 치려야 할 때.
어떻게 쓰나	대역이 구역 하나 안에 엄격히 들어가도록 f_s 를 고르고($k \cdot f_s/2$ 에서 $(k+1) \cdot f_s/2$ 까지), 대역통과 와 안티앨리어스 필터를 단다 — 여기서 저역통과는 쓸모가 없다 — 그리고 지터를 RF 반송파에 견주어 예산 잡는다.

실제 코드는 이렇게 생겼다

```
// Zone check, written as an assertion rather than a comment.
// A band that straddles a zone boundary is a silent failure: it still samples,
// and the output is two overlapped copies.
localparam real FS = 250e6, FC = 2.4e9, BW = 100e6;
initial begin
    int k = $floor(2.0*(FC - BW/2)/FS);
    assert ($floor(2.0*(FC + BW/2)/FS) == k)
        else $fatal(1, "band straddles a Nyquist zone boundary -- pick another FS");
end
```

여기서 틀린다. 대역통과 표본화의 안티앨리어스 필터는 $f_s/2$ 위의 주파수만이 아니라 **다른 모든 구역을** 막아야 한다. 여기에 저역통과 필터를 재활용하는 기술자는 실험실에서는 되고(거기에는 원하는 신호밖에 없다) 현장에서는 안 되는 수신기를 얻는다.

T5.2 구경 지터 — 벽

구경지터 (aperture jitter) — 표본화기가 신호를 실제로 잡는 순간의 rms 불확실도. 클럭 경로(위상 잡음, 전원이 일으키는 지연 변조, 마지막 버퍼의 열잡음)와 표본화기 자신에게서 오고, 표본화율이 아니라 **입력 주파수에 환산**된다.

유도 — 신호가 빨라질수록 지터가 왜 더 비싸지나

	이렇게 된다	왜냐하면
1	사인파 $v(t) = A \cdot \sin(2\pi f_{in} t)$ 의 기울기는 $dv/dt = 2\pi f_{in} A \cdot \cos(\dots)$ 다.	기울기가 시간 오차를 전압 오차로 바꾸는 것이다 — 기구가 그것뿐이다.
2	그러므로 시간 오차 Δt 는 전압 오차 $\Delta v = (dv/dt) \cdot \Delta t$ 를 놓고, 기울기가 가장 큰 영교차에서 가장 나쁘다.	가장 큰 오차는 신호가 가장 클 때가 아니라 가장 빨리 변할 때 난다.
3	rms 지터 σ_t 와 전체 눈금 사인파에서 rms 오차는 $2\pi f_{in} \cdot (A/\sqrt{2}) \cdot \sigma_t$ 이고, 신호 rms 는 $A/\sqrt{2}$ 다.	진폭이 사라진다 — 지터가 묶는 SNR 은 신호 진폭과 무관하다 . 크기를 줄여서 거기서 물러날 수 없는 까닭이다.
4	$SNR = -20 \cdot \log_{10}(2\pi f_{in} \sigma_t)$.	식 하나, 변수 둘, 소자 매개변수는 없다. 오로지 클럭과 입력 주파수만의 성질이다.
5	$f_{in} = 1 \text{ GHz}$ 에서 지터 100 fs 면 64 dB — 10.3 비트다.	100 fs 는 이미 어려운 클럭이다 . 기가헤르츠 12 비트 변환기가 비교기와는 아무 상관 없는 방식으로 어려운 까닭이다.

표 86. 지터가 묶는 SNR 과 ENOB (서로 무관한 구경 지터만)

f_{in}	1 ps	300 fs	100 fs	30 fs
0.1 GHz	64.0 dB / 10.3 b	74.5 dB / 12.1 b	84.0 dB / 13.7 b	94.5 dB / 15.4 b
1 GHz	44.0 dB / 7.0 b	54.5 dB / 8.8 b	64.0 dB / 10.3 b	74.5 dB / 12.1 b
5 GHz	30.1 dB / 4.7 b	40.5 dB / 6.4 b	50.1 dB / 8.0 b	60.5 dB / 9.8 b

예제 14. 입력 1 GHz 12 비트 변환기에는 어떤 클럭이 필요한가?

주어진 것. 설계 목표는 입력 1 GHz 에서 실효 12 비트다. 지터가 유일한 오차원이라고 하자 — 낙관적인 가정이고, 그래서 답은 클럭이 얼마나 좋아야 하는가의 **하한**이 된다.

푸는 법. SNR 식을 뒤집는다: $\sigma_t = 1/(2\pi \cdot f_{in} \cdot 2^N)$.

답. $\sigma_t = 38.9$ fs rms. 14 비트면 9.71 fs 다.

표 87. $f_{in} = 1$ GHz 에서 필요한 rms 구경 지터

비트	필요한 σ_t
8	622 fs
10	155 fs
12	38.9 fs
14	9.71 fs
16	2.43 fs

여기서 틀린다. 그 수를 클럭 소스에 적힌 지터와 건주는 것. 중요한 규칙은 분배 경로 전체 — 버퍼, 레벨 시프터, 분주기, 그리고 디지털 쪽이 거기에 집어넣은 전원 잡음 — 를 지난 뒤 **표본화 스위치에서의** 지터다. 시끄러운 버퍼 사슬 뒤에 있는 20 fs 소스는 200 fs 표본화기이고, 소스의 데이터시트는 그것을 말해 주지 않는다.

지터와 kT/C 는 진폭이 아니라 **전력**으로 더해지고, 주파수에 대해 서로 반대로 비례한다. kT/C 는 평평한데 지터는 입력 주파수 한 옥타브마다 6 dB 씩 나빠진다. 그러므로 갈림 주파수가 늘 있고, 자신이 그 어느 쪽에 있는지를 아는 것이 어느 손잡이가 무언가를 하는지 말해 준다. 아래쪽이면 면적을 용량에 쓰라. 위쪽이면 전력을 클럭에 쓰라 — **지터가 묶는 설계에서 표본화 축전기를 키우는 것은 아무것도 사지 못하고, 재설계를 낭비하는 가장 흔한 길이다.**

T5.3 스위치 자체: 차지 인젝션, 피드스루, 그리고 부트스트랩

VDD

그림 35. 표본화 스위치. 그 온저항이 트래킹 대역폭을 정하고, 그 채널 전하는 스위치가 꺼질 때 표본화 축전기에 얹힌다.

표 88. 스위치가 더하는 오차 셋, 그리고 각각이 무엇에 달려 있나

오차	기구	무엇에 비례하나	신호 의존?
차지 인젝션	소자가 꺼질 때 채널의 반전 전하가 어디론가 가야 한다. 대략 절반이 C_s 에 얹힌다	$W \cdot L \cdot C_{ox} \cdot (V_{GS} - V_{th}) / C_s$	그렇다 — V_{GS} 가 표본화된 준위에 달려 있으므로 오프셋이 아니라 왜곡이 된다
클럭 피드스루	C_{gd} 가 떨어지는 클럭 에지를 축전기로 결합시킨다	$C_{ov} / (C_{ov} + C_s) \cdot V_{clk}$	아니다 (1 차로) — 대체로 고정 오프셋이다
R_{on} 변조	스위치의 온저항이 $V_{GS} = V_{clk} - v_{in}$ 에 달려 있다	$1/(\mu C_{ox} (W/L) (V_{GS} - V_{th}))$	그렇다 — 신호에 따라 변하는 시간상수는 신호에 따라 변하는 정착이고, 그것이 왜곡이다

부트스트랩 스위치

스위치의 게이트를 V_{DD} 가 아니라 $v_{in} + V_{DD}$ 에 붙들어 둔다. V_{DD} 로 미리 충전한 축전기를 트래킹 구간에서 게이트로 뒤집어 붙여서 그렇게 한다. 그러면 V_{GS} 가 신호와 무관하게 일정하므로 R_{on} 이 일정하고 꺼질 때 주입되는 전하도 일정하다 — **신호에 따라 변하던 오차가 오프셋이 되고**, 차동 설계에서 오프셋은 해롭지 않다.

놓치기 쉬운 신뢰성 문제도 같이 풀린다. 부트스트랩이 없으면 접지 근처 신호를 지나보내는 스위치가 $V_{GS} = V_{DD}$ 를 보게 되는데, 진보한 노드에서는 그것이 소자 수명에 걸쳐 산화막의 정격을 넘을 수 있다(T14).

어디에 쓰나	10 비트를 넘는 사실상 모든 SAR 과 파이프라인 ADC 의 입력 표본화 회로망, 그리고 선형성이 높은 스위치드 커패시터 필터.
언제 쓰나	면적보다 THD 나 SFDR 이 중요할 때. 맨 NMOS 스위치는 8 비트에는 괜찮고 12 비트에는 가망이 없다.
어떻게 쓰나	부트스트랩 축전기를 그것이 몰아야 할 게이트 용량보다 훨씬 크게 잡고, 클럭 구간마다 모든 마디의 신뢰성을 확인하고, 왜곡을 단일 톤이 아니라 두 톤 과도로 검증한다 — 상호변조가 단일 톤이 감추는 잔여 신호 의존을 드러낸다.

실제 코드는 이렇게 생겼다

```
* The check that matters, and the one people skip.
* A single tone can look clean while IM3 is terrible.
Vin1 in1 0 SIN(0 0.45 100.1MEG)
Vin2 in2 0 SIN(0 0.45 103.3MEG)
.tran 10p 20u
.four 100.1MEG v(sampled)          $ harmonic distortion
* and post-process the FFT for IM3 at 2*f1-f2 = 96.9 MHz -- the number the
* customer will measure and the one that exposes Ron modulation.
```

여기서 틀린다. 부트스트랩은 게이트 전압을 전원 위로 올리므로, 부트스트랩 회로망의 모든 마디를 **빠르고, 전압이 높고, 온도가 낮은** 코너에서 산화막 한계에 견주어 확인해야 한다 — 성능을 묶는 코너와 반대쪽 코너다. 전형 코너에서만 검증한 부트스트랩 회로는 신뢰성 보고서가 기다리고 있는 것이다.

T5.4 오버샘플링: 시간으로 비트를 사기

유도 — 오버샘플링이 왜 도움이 되고, 얼마나 되나

	이렇게 된다	왜냐하면
1	양자화 잡음 전력은 표본화율과 무관하게 $LSB^2/12$ 다.	양자화기는 자기가 얼마나 빨리 클럭되는지 모른다. 표본마다의 오차는 같은 분포를 갖는다.
2	그 고정된 전력이 나이퀴스트 대역 0 에서 $f_s/2$ 에 펼쳐진다.	그래서 잡음 밀도는 $LSB^2/12$ 를 $f_s/2$ 로 나눈 것이다 — 빨리 표본화하면 뭉어진다.
3	신호 대역 B 까지 걸러 내면 그중 $2B/f_s$ 만 남는다.	오버샘플링비를 $OSR = f_s/(2B)$ 로 정의한다.
4	SNR 이 $10 \cdot \log_{10}(OSR)$ 만큼 좋아진다: 두 배마다 3 dB, 곧 반 비트.	되먹임이 없는 맨 오버샘플링이다. 이 방식으로 8 비트에서 12 비트로 가려면 $OSR = 256$ 이 필요하다 — 비싸지만 느린 센서에는 가끔 맞는 답이다.
5	$\Delta\Sigma$ 변조기는 잡음을 그저 펼치는 것이 아니라 모양 지운다 . L 차 고리면 두 배마다 $(6L+3)$ dB 다.	고리가 양자화 잡음을 대역 밖으로 밀어낸다. 2 차 변조기는 OSR 한 옥타브마다 15 dB 를 번다 — $\Delta\Sigma$ 가 오디오와 정밀 감지를 지배하는 까닭이고, 기가 헤르츠 신호 대역폭에서는 쓸모가 없는 까닭이다.

표 89. OSR 을 두 배로 할 때마다 오버샘플링이 사 주는 것

구조	OSR 2 배당 SNR 이득	OSR 2 배당 비트	어디서 이기나
맨 오버샘플링 + 데시메이션	3 dB	0.5	느린 신호에서 값싼 해상도. 안티앨리어스 필터 부담 덜기
$\Delta\Sigma$, 1 차	9 dB	1.5	간단하다. 다만 유희 톤이 있다
$\Delta\Sigma$, 2 차	15 dB	2.5	오디오, 정밀 센서
$\Delta\Sigma$, 3 차 이상 / MASH	21 dB+	3.5+	안정성 공학을 치를 값어치가 있는 자리

오버샘플링은 해상도를 사 주는 만큼이나 **안티앨리어스 필터**의 부담을 덜어 주고, 흔히 그것이 진짜 이유다. 20 kHz 오디오 대역을 44.1 kHz 로 표본화하려면 20 kHz 에서 평평하고 22.05 kHz 에서 죽은 필터가 있어야 한다 — 잔인한 아날로그 문제다. 2.8 MHz 로 표본화하면 요구가 '1.4 MHz 에서 죽어라' 로 옮겨 가고, 그것은 RC 하나다. **사라지는 그 필터가 대개 반 비트보다 값어치가 크다.**

T5.5 월요일에 무엇을 할 것인가

표 90. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
엘리어싱 · 나이키스트	f_s 와 안티엘리어스 필터 고르기	필터 차수 — 그리고 그것이 아날로그인지 오버샘플링과 나눠 질 수 있는지
구경지터	변환기를 설계하기 전의 클럭 구조	그 입력 주파수에서 목표 ENOB 에 닿는 것이 가능하거나 한가
대역통과표본화	RF 와 IF 수신기 전단	믹서를 뺄 수 있는가, 그리고 그러면 클럭에 얼마를 치러야 하는가
차지인젝션 · 부트스트랩 스위치	10 비트 이상 변환기의 표본화 회로망	THD 와 SFDR, 그리고 그에 딸린 산화막 신뢰성 검사
오버샘플링비 · 잡음성형	느리고 정밀한 신호 경로	용량 대신 시간으로 해상도를 살 수 있는가

들고 갈 수: 입력 1 GHz 에서 **실효 12 비트는 rms 구경 지터 38.9 fs 를 요구한다**. 클럭 소스가 아니라 스위치에서 낸 값으로. 고속 변환기의 모든 구조 결정 — 인터리빙, 대역통과 표본화, 클럭 버퍼가 어디에 앉는가 — 이 그 한 줄의 아래에 있다.

다음 장은 표본화된 전압을 코드로 바꾸고, 구조들이 정확도에서 다른 것보다 같은 근본 한계를 어디에 쓰는지에서 더 다르다는 것을 알아낸다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

표본화 순간은 고속 변환기에서 가장 많이 다투는 1 제곱 밀리미터이고, 특허가 그것을 보여 준다. 온저항이 신호에 안 달리도록 V_{GS} 를 일정하게 붙드는 부트스트랩 스위치. 다른 무엇과도 버퍼를 일부러 안 나누는 클럭 경로. 트리밍이 아니라 대칭으로 차지 인젝션을 상쇄하는 표본화 방식. 그리고 한 단계 위에서, 조각마다 조금씩 틀린 순간에 표본화한다는 것이 어려움의 전부인 시간 인터리브 구조. **그 하나하나가 증폭기가 아니라 구경에 대한 논의다.**